

A.QEMU 使用手册

1. QEMU 简介

QEMU 的英文单词是：Quick Emulator，它是一个小巧的模拟器。还有很多模拟器，比如 VMWare、Virtual Box 等。但是 VMWare、Virtual Box 只能模拟 x86、AMD64/Intel64 等 PC 系统；而 QEMU 可以模拟更多硬件：ARM、MIPS、PPC、x86、AMD64/Intel64。

QEMU 用途广泛，比如 Xen、Android 模拟器等都是基于 QEMU 的。

在嵌入式领域，很多人使用 QEMU 来深研 Linux，比如研究文件系统、优化等等。

QEMU 有两种模式：

1.1 用户模式 (User Mode)

简单地说，一个使用 arm-xxx-gcc 编译出来的程序，是给 ARM 板子使用的，它无法在 PC 机上运行，只能放到 ARM 板子上去运行。

借助 qemu，可以在 PC 机上运行 ARM 程序。比如：

```
$ gcc -o hello hello.c -static
$ ./hello      // 这个 hello 程序是使用 gcc 给 PC 机编译的，可以直接运行
Hello, world!
$ arm-linux-gnueabi-gcc -o hello hello.c -static // 它是给 ARM 板子编译的
$ ./hello      // 所以无法在 PC 上运行
bash: ./hello: cannot execute binary file: Exec format error
$ ./qemu-arm ./hello      // 我们可以用 QEMU 在 PC 上运行它
Hello, world!
```

在 PC 上使用 qemu 运行单个 ARM 程序时，这就是使用 QEMU 的用户模式。

它会把 ARM 指令翻译为 PC 的指令去运行。

注意，你可能无法做上述实验，因为：

- 你没有安装 ARM 交叉编译工具链
- 你没有安装 QEMU

你根据下章《QEMU 快速使用》安装 QEMU 后，就可以进行上述实验了。

1.2 系统模式 (System Mode)

很多时候我们并不满足于在 PC 上运行单个 ARM 程序，我们想模拟出整个 ARM 单板：在这个模拟出来的虚拟 ARM 单板上，运行 Linux 系统，在其中运行各种 APP。

这时候需要使用 QEMU 的系统模式。

我们就是使用 QEMU 的系统模式来模拟 IMX6ULL 开发板，具体的使用请看下章。

1.3 我们做的改进

QEMU 可以模拟 x86，也可以模拟各种 ARM 板子，还可以模拟各种外设。

百问网对 QEMU 做了很多改进，支持更多硬件，支持更多 GUI 现实，支持更方便的调试。

- 100ask-qemu 特点
 1. 模拟网卡
 2. 模拟 LCD 显示功能
 3. 模拟 led 灯、按键
 4. 模拟 at24cxx i2c 存储芯片，直接可以通过用户态操作看到效果
 5. 增加逻辑分析仪显示功能
 6. 后续会逐渐增加更多的模拟硬件模块
 - 6.1 温湿度传感器
 - 6.2 红外
 - 6.3 超声波模块
 - 6.4 ADC,DAC 模块
 - 6.5 I2C 接口的传感器
 - 6.6 SPI 接口的 OLED
 - 6.7 你想加啥，跟我们说

2. QEMU 快速使用

使用 apt-get 当然也可以安装 QEMU，但是它版本太低，也不支持 IMX6ULL。
新版本 QEMU 已经支持 IMX6ULL，我们在此基础上添加了更多功能，也修改了一些 BUG。
所以，请参考本文安装我们定制的 QEMU。

2.1 准备工作

2.1.1 Windows 电脑和 VMWare 虚拟机

必须的：

- ① 一台可以上网的 windows 电脑
- ② 一个可 VMWare 虚拟机，里面运行 Ubuntu，也要能上网

VMWare 虚拟机软件和 Ubuntu 映象可以从网盘中下载，打开以下链接：
http://wiki.100ask.org/Download_link_page
找到“韦东山升级版全系列视频开发板 BSP 包”，打开对应的网盘链接后，可以看到一个目录“100ask_imx6ull-qemu”，进入“01_Tools”子目录：



2.1.2 确保 Ubuntu 可以上网，可以跟 Windows 联通

注意：很多人碰到 VMWare 中 Ubuntu 联网问题问题，可以参考此链接：
<http://wiki.100ask.org/VMwareAndUbuntuNetworkSetupGuide>

2.1.3 安装 KVM

可选的(注意，如果想要有更快的效果，在 ubuntu 下可以安装 KVM)：
安装 kvm 加速 qemu 运行，在终端下执行如下命令：

```
$ sudo apt-get update
$ sudo apt-get install qemu qemu-kvm libvirt-bin bridge-utils virt-manager
```

2.2 获取镜像

2.2.1 下载和解压

我们只提供 ubuntu 16.04 和 ubuntu 18.04 系统的 QEMU 镜像文件。

- ① ubuntu-18.04 开发环境下 qemu imx6ul 系统镜像下载地址：

http://wiki.100ask.org/Download_ubuntu-18.04_imx6ul_qemu

- ② ubuntu-16.04 开发环境下 qemu imx6ul 系统镜像下载地址：

http://wiki.100ask.org/Download_ubuntu-16.04_imx6ul_qemu

下载完成后上传到 Ubuntu 虚拟机中，执行如下命令进行解压缩操作。

- ① ubuntu-16.04 解压操作步骤

```
$ tar -xvf weidongshan-ubuntu-16.04_imx6ul_qemu_system-release.tgz
```

然后进入 ubuntu-16.04_imx6ul_qemu_system 目录执行后文介绍的命令。

- ② ubuntu-18.04 解压操作步骤

```
$ tar -xvf weidongshan-ubuntu-18.04_imx6ul_qemu_system-release.tgz
```

然后进入 ubuntu-18.04_imx6ul_qemu_system 目录执行后文介绍的命令。

2.2.2 镜像目录结构

目录结构及说明如下，在后续的开发过程中，我们有可能更换红框中的文件：

```

├── imx6ull-system-image // 镜像文件
│   ├── 100ask_imx6ull_qemu.dtb // 设备树文件
│   ├── zImage // 内核镜像文件
│   └── rootfs.img // 文件系统镜像文件
├── qemu-imx6ull-gui.sh // 显示GUI的运行脚本
├── qemu-imx6ull-nogui.sh // 不显示GUI的运行脚本
├── qemu // qemu模拟器软件
│   ├── bin // bin文件，里面含有模拟器软件qemu-system-arm
│   └── etc // 板子图片及生成的逻辑分析仪文件
├── README.en // 英文版使用介绍
├── README.zh // 中文版使用介绍
├── install_sdl.sh // sdl软件包一键安装脚本。
└── ubuntu-18.04_sdl-package // sdl依赖包

```

2.3 运行 QEMU 系统

假设你已经按照上文下载、解压好了 QEMU 镜像文件，你需要进入 QEMU 的目录，执行下列命令。

2.3.1 首次运行需要安装 SDL 环境以及解压文件系统镜像:

使用脚本自动解压安装:

```
$ ./install_sdl.sh // 提示输入用户密码，等待安装完成
```

2.3.2 运行带 GUI 的 imx6ul 模拟器

① 模拟百问网 imx6ull-qemu 开发板

```
$ ./qemu-imx6ull-gui.sh // 启动后，登录名是 root，无需密码
```

② 模拟野火 imx6ull-pro 开发板

```
$ ./qemu-imx6ull-gui.sh fire // 启动后，登录名是 root，无需密码
```

③ 模拟正点原子 imx6ull-alpha 开发板

```
$ ./qemu-imx6ull-gui.sh atk // 启动后，登录名是 root，无需密码
```

③ 运行不带 GUI 的 imx6ull 模拟器

```
$ ./qemu-imx6ull-nogui.sh // 启动后，登录名是 root，无需密码
```

2.3.3 参数讲解

可以打开脚本文件 qemu-imx6ull-gui.sh，它就是运行 qemu-system-arm 程序。其中用到了很多参数：

-M mcimx6ul-evk	指定需要模拟的单板型号。
-m 512M	指定板子的内存大小。
-kernel zImage	指定使用的内核镜像文件。
-dtb 100ask_imx6ull_qemu.dtb	指定使用的设备树文件。
-display sdl	指定使用那种图形显示输出。
-serial mon:stdio	指定串口信息输出。
-drive file=rootfs.img,format=raw,id=mysdcard	名为 mysdcard 的 drive,源为 rootfs.img
-device sd-card,drive=mysdcard	添加一个 sd-card 设备,内容来自名为 mysdcard 的 drive
-append "console=ttyMXC0,115200 rootfstype=ext4 root=/dev/mmcblk1 rw rootwait init=/sbin/init loglevel=8"	指定内核的命令行参数
-nic user	指定网卡为 user mode

有了内核 zImage、设备树、文件系统(rootfs.img)，这就是一个完整的 Linux 系统。

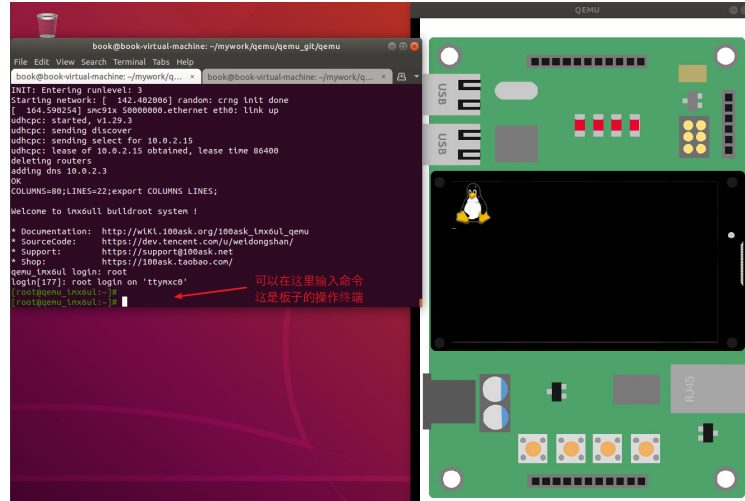
注意：QEMU 中没有实现 bootloader，以后我们会完全模拟 SD 卡，在 SD 卡上面放置 u-boot、内核、设备树、文件系统。

2.4 QEMU 操作示例

先执行以下命令启动 QEMU，它模拟百问网 imx6ull-qemu 开发板：

```
$ ./qemu-imx6ull-gui.sh // 启动后，登录名是 root，无需密码
```

它会弹出一个开发板的界面，并且运行该开发板的 Linux 系统，你可以在终端中操作该开发板：



注意：当你的鼠标点击 QEMU 的 GUI 界面时，鼠标将无法移出这个 GUI 界面。这时可以通过快捷键“Ctrl+Alt+g”把鼠标从 GUI 界面中退出来。

2.4.1 操作设备管理器

我们的计划是模拟更多的外设，为形象地操作这些外设，每个外设都会有一个 GUI 界面。可能会有多达几十、上百个外设，如果一下子显示那么多 GUI 界面，会很乱。

所以我们实现了一个“设备管理”，如下所示。可以点击它上面的某个按钮，显示或隐藏某个外设的 GUI 界面。

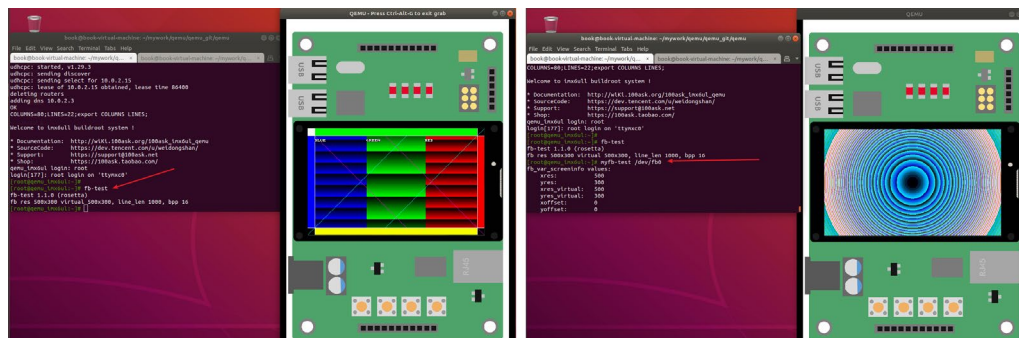


2.4.2 操作 LCD

我们模拟的 IMX6ULL 板子，它的 Linux 系统中已经带有 LCD 的测试命令，可以执行以下命令测试：

```
[root@qemu_imx6ul:~]# fb-test
或
[root@qemu_imx6ul:~]# myfb-test /dev/fb0
```

效果如下：

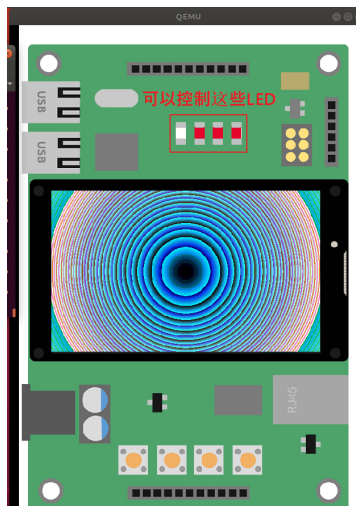


2.4.3 操作 LED

我们模拟的 IMX6ULL 板子，它的 Linux 系统中已经带有 LED 驱动和测试命令，可以执行以下命令测试：

```
[root@qemu_imx6ul:~]# cd led_driver_qemu/
[root@qemu_imx6ul:~/led_driver_qemu]# insmod 100ask_led.ko
[root@qemu_imx6ul:~/led_driver_qemu]# ./ledtest /dev/100ask_led0 off
[root@qemu_imx6ul:~/led_driver_qemu]# ./ledtest /dev/100ask_led0 on
```

效果如下：



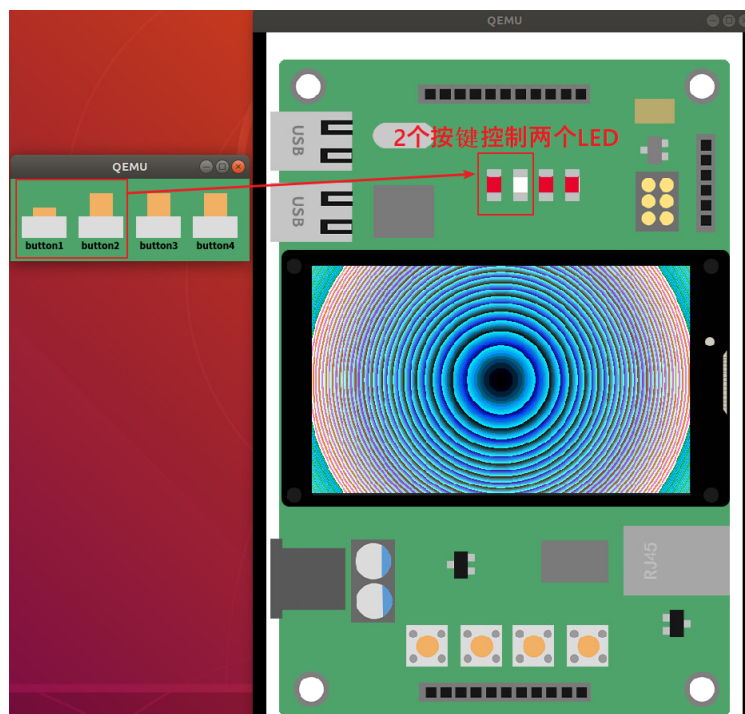
2.4.4 使用按键来控制 LED

我们模拟的 IMX6ULL 板子，它的 Linux 系统中已经带有 LED 驱动、按键驱动和测试命令。

首先在“QEMU 设备管理器”中打开按键的界面，然后执行以下命令测试：

```
[root@qemu_imx6ul:~]# cd led_driver_qemu/  
[root@qemu_imx6ul:~/led_driver_qemu]# insmod 100ask_led.ko  
[root@qemu_imx6ul:~/led_driver_qemu]# cd ../button_driver_qemu/  
[root@qemu_imx6ul:~/button_driver_qemu]# insmod button_drv.ko  
[root@qemu_imx6ul:~/button_driver_qemu]# insmod board_100ask_qemu_imx6ull.ko  
[root@qemu_imx6ul:~/button_driver_qemu]# ./button_led_test
```

效果如下图(只实现了左边的 2 个按键，右边的 2 个按键留待以后实现其他目的)：



2.4.5 读写 I2C EEPROM AT24C02

我们模拟的 IMX6ULL 板子，它已经实现了对 I2C 设备 AT24C02 的模拟。

首先在“QEMU 设备管理器”中打开 at24c02 的界面，然后执行以下命令测试：

// 0x50 是 AT24C02 的 I2C 设备地址

```
[root@qemu_imx6ul:~]# i2c_usr_test /dev/i2c-0 0x50 r 0 // 读地址 0
```

data: , 0, 0x00

```
[root@qemu_imx6ul:~]# i2c_usr_test /dev/i2c-0 0x50 w 1 0x58 // 写地址 1, 写入 0x58
```

效果如下图：

绿色表示读 红色表示写

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
00h	00	58	00	00	00	00	00	00	00	00	00	00	00	00	00	00
10h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
20h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
30h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
50h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
60h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
70h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
80h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
90h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
a0h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
b0h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
c0h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
d0h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
e0h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
f0h	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

3. 使用 QEMU 进行嵌入式 Linux 开发

完整的嵌入式 Linux 系统包含：bootloader、Linux 内核、设备树、根文件系统。

QEMU 可以略过 bootloader 而直接启动内核，这给开发带来了便利。

但是以后我们会修改 QEMU，让它支持从 SD 卡上启动 bootloader，让 bootloader 来启动内核。

在本章中，我们介绍如何编译并替换内核、编译并替换设备树，修改根文件系统。

3.1 获取源码

我们的源码都是存放在 GIT 中，所以需要确保你的 Ubuntu 可以上网。如果不能上网，请参考以下链接：

<http://wiki.100ask.org/VMwareAndUbuntuNetworkSetupGuide>

3.1.1 设置 git 邮箱账号和用户名

在 Ubuntu 中执行如下命令：

```
book@100ask:~$ git config --global user.email "you@example.com"
book@100ask:~$ git config --global user.name "Your Name"
```

初次使用 GIT 时，需要配置邮箱帐号和用户名，可以随意指定。

3.1.2 下载源码

考虑到代码仓库过多，特使用 repo 工具管理代码。

先用 git clone 下载 repo 工具，再用 repo 工具下载源码：

```
book@100ask:~$ git clone https://git.dev.tencent.com/codebug8/repo.git
book@100ask:~$ mkdir -p 100ask_imx6ull-qemu && cd 100ask_imx6ull-qemu
book@100ask:~/100ask_imx6ull-qemu$ ../repo/repo init -u
https://dev.tencent.com/u/weidongshan/p/manifests/git -b linux-sdk -m
imx6ull/100ask-imx6ull-qemu_release_v1.0.xml --no-repo-verify
book@100ask:~/100ask_imx6ull-qemu$ ../repo/repo sync -j4
```

上面使用的 repo 管理的是国内 coding 仓库，从国内仓库下载会快很多。

如果一切正常，你在/home/book 目录下创建了一个 100ask_imx6ull-qemu 目录，里有如下内容：

```
book@book-virtual-machine:~/100ask_imx6ull-qemu$ ls
buildroot2019.02  linux-4.9.88  qemu  ToolChain
```

3.2 百问网 imx6ull-qemu 开发板资料下载

百问网 imx6ull-qemu 开发板，并不是真实的开发板，我们可以在它上面添加任意硬件，当然也会提供原理图。

这款虚拟开发板的资料可以从网盘中下载，打开以下链接：

http://wiki.100ask.org/Download_link_page

找到：找到“**韦东山升级版全系列视频开发板 BSP 包**”，打开对应的网盘链接后，可以看到一个目录“100ask_imx6ull-qemu”。

3.3 设置工具链

交叉编译工具链主要是用于在 ubuntu 主机上编译可以在其它平台上运行的系统，比如在 PC 上为 ARM 板子编译程序。

设置交叉编译工具主要是设置 PATH， ARCH 和 CROSS_COMPILE 三个环境变量，下面介绍具体设置方法。

3.3.1 永久生效

如需永久修改，请修改用户配置文件。在 Ubuntu 系统下，修改如下：

```
book@100ask:~$ vi ~/.bashrc
```

在行尾添加或修改：

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
export
PATH=$PATH:/home/book/100ask_imx6ull-qemu/ToolChain/gcc-linaro-6.2.1-2016.11-x86_64_arm-linux-gnueabihf/bin
```

3.3.2 临时生效

如果你有很多单板，为了不冲突，你不能使用“永久生效”的办法。比如你有 32 位的 ARM 板，也有 64 位的 ARM 板，在使用前者时需要设置 ARCH=arm，在使用后者时需要设置 ARCH=arm64。

这种情况下，可以使用“export”命令设置环境变量，这种设置方法只对当前终端有效：

```
book@100ask:~$ export
PATH=$PATH:/home/book/100ask_imx6ull-qemu/ToolChain/gcc-linaro-6.2.1-2016.11-x86_64_arm-linux-gnueabihf/bin
book@100ask:~$ export ARCH=arm
book@100ask:~$ export CROSS_COMPILE=arm-linux-gnueabihf-
```

3.3.3 手动指定

执行 make 命令时，可以手工指定 ARCH 架构、CROSS_COMPILE 等变量：

```
book@100ask:~$ export  
PATH=$PATH:/home/book/100ask_imx6ull-qemu/ToolChain/gcc-linaro-6.2.1-2016.11-x86  
_64_arm-linux-gnueabihf/bin  
book@100ask:~$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-
```

不过这种方法效率太低，不建议使用。

3.4 编译内核及设备树

前面我们下载了源码，设置好工具链后，即可编译：

```
book@100ask:~/100ask_imx6ull-qemu$ cd linux-4.9.88  
book@100ask:~/100ask_imx6ull-qemu/linux-4.9.88$ make mrproper  
book@100ask:~/100ask_imx6ull-qemu/linux-4.9.88$ make 100ask_imx6ull_qemu_defconfig  
book@100ask:~/100ask_imx6ull-qemu/linux-4.9.88$ make zImage -jN //编译 zImage 内核  
镜像，其中 N 参数可以根据 CPU 个数，来加速编译系统。  
book@100ask:~/100ask_imx6ull-qemu/linux-4.9.88$ make dtbs //编译设备树文件
```

编译成功后，可以得到如下文件：

```
arch/arm/boot/zImage // 内核  
arch/arm/boot/dts/100ask_imx6ull_qemu.dtb // 设备树
```

如果你修改过内核，或是修改过设备树文件，那么可以用上面 2 个文件去替换 QEMU 中的 zImage 和 100ask_imx6ull_qemu.dtb。

QEMU 中的 zImage 和 100ask_imx6ull_qemu.dtb 在哪？你安装我们提供的 QEMU 时，可以得到这样的脚本：qemu-imx6ull-gui.sh。打开它就可以知道这 2 个文件在哪里了。

一般位于 imx6ull-system-image 目录下：

```
book@100ask:~/Desktop$ cd ubuntu-18.04_imx6ul_qemu_system-release/  
book@100ask:~/Desktop/ubuntu-18.04_imx6ul_qemu_system-release$ ls  
imx6ull-system-image qemu-imx6ull-gui.sh ubuntu-18.04_sdl-package  
install_sdl.sh qemu-imx6ull-nogui.sh  
qemu README.md  
book@100ask:~/Desktop/ubuntu-18.04_imx6ul_qemu_system-release$ ls imx6ull-system-image/  
100ask_imx6ull_qemu.dtb rootfs.img rootfs.tar.gz zImage  
book@100ask:~/Desktop/ubuntu-18.04_imx6ul_qemu_system-release$
```

3.5 修改文件系统

安装好我们提供的 QEMU 后，你可以得到一个 imx6ull-system-image 目录，里面有名为 rootfs.img 的文件，它就是根文件系统：

```
book@100ask:~/Desktop$ cd ubuntu-18.04_imx6ul_qemu_system-release/  
book@100ask:~/Desktop/ubuntu-18.04_imx6ul_qemu_system-release$ ls  
imx6ull-system-image  qemu-imx6ull-gui.sh  ubuntu-18.04_sdl-package  
install_sdl.sh        qemu-imx6ull-nogui.sh  
qemu                  README.md  
book@100ask:~/Desktop/ubuntu-18.04_imx6ul_qemu_system-release$ ls imx6ull-system-image/  
100ask_imx6ull_qemu.dtb  rootfs.img  rootfs.tar.gz  zImage  
book@100ask:~/Desktop/ubuntu-18.04_imx6ul_qemu_system-release$
```

你可以在 Ubuntu 下直接修改 rootfs.img，不过要先挂载，执行以下命令：

```
book@100ask:~/Desktop/ubuntu-18.04_imx6ul_qemu_system-release$ ls imx6ull-system-image/  
100ask_imx6ull_qemu.dtb  rootfs.img  rootfs.tar.gz  zImage  
book@100ask:~/Desktop/ubuntu-18.04_imx6ul_qemu_system-release$ cd imx6ull-system-image/  
book@100ask:~/Desktop/ubuntu-18.04_imx6ul_qemu_system-release/imx6ull-system-image$ sudo mou  
mount mount.lowntfs-3g mount.ntfs4 mount.ntfs-3g mountstats mousetweaks  
mount.fuse mount.nfs mount.ntfs mountpoint mount.vmhgfs  
book@100ask:~/Desktop/ubuntu-18.04_imx6ul_qemu_system-release/imx6ull-system-image$ sudo mount -o loop rootfs.img /mnt  
[sudo] password for book:  
book@100ask:~/Desktop/ubuntu-18.04_imx6ul_qemu_system-release/imx6ull-system-image$ ls /mnt  
bin boot dev etc fat home lib lib32 linuxrc media mnt opt proc root run samba/sbin sys usr var  
book@100ask:~/Desktop/ubuntu-18.04_imx6ul_qemu_system-release/imx6ull-system-image$
```

主要命令就是：

```
sudo mount -o loop rootfs.img /mnt
```

你就可以在 /mnt 目录下操作其中的文件了，也可以把 Ubuntu 中的文件复制进去。

注意：修改完毕后，要执行以下命令：

```
sudo umount /mnt
```

3.6 启动模拟器后使用 NFS

安装好我们提供的 QEMU 后，可以执行如下命令启动开发板：

```
$ ./qemu-imx6ull-gui.sh // 启动后，登录名是 root，无需密码
```

等待进入系统后，就可以使用网络命令挂载 Ubuntu 的 NFS 目录了。

3.6.1 在 Ubuntu 上安装、配置 NFS 服务

如果你使用的是我们提供的 Ubuntu，那么已经安装好了 NFS 服务。

如果你的 Ubuntu 未安装 NFS 服务，在确保 Ubuntu 可以上网的前提下，执行以下命令：

```
sudo apt-get install nfs-kernel-server
```

然后，还得修改 `/etc/exports`，添加类似以下的内容，下面的例子里允许开发板通过 NFS 访问 Ubuntu 的 `/home/book`、`/work` 两个目录：

```
/home/book *(rw,nohide,insecure,no_subtree_check,async,no_root_squash)  
/work *(rw,nohide,insecure,no_subtree_check,async,no_root_squash)
```

最后，重启 NFS 服务，在 Ubuntu 上执行以下命令：

```
sudo /etc/init.d/nfs-kernel-server restart
```

可以在 Ubuntu 上通过 NFS 挂载自己，验证一下 NFS 可用：

```
sudo mount -t nfs -o nolock,vers=3 127.0.0.1:/home/book /mnt  
ls /mnt
```

3.6.2 开发板获取 IP 地址

QEMU 运行时, Ubuntu 是 Host 即宿主机, QEMU 给它分配的 IP 是 10.0.2.2。
QEMU 模拟的 imx6ull 板子是 Guest 即客户机, 它会自动获取 IP, 也可以自己设置。
Guest 可以通过 10.0.2.2 访问 Host, Host 不能访问 Guest。
Guest 中可以使用 ifconfig 命令查看 IP, 如果没有 IP, 可以如下设置:

```
[root@qemu_imx6ul:~]# ifconfig eth0 10.0.2.15
```

3.6.3 挂载主机 nfs 目录

QEMU 模拟的 imx6ull 开发板, 可以去访问 10.0.2.2, 比如使用 NFS 挂载:

```
[root@qemu_imx6ul:~]# mount -t nfs -o nolock,vers=3 10.0.2.2:/home/book/nfs_rootfs  
/mnt
```

如果一切正常, 在开发板上就可以通过/mnt 目录访问 Ubuntu 的/home/book/nfs_rootfs 目录了。

4. 更新 QEMU

我们使用 QEMU 来模拟 IMX6ULL 开发板，目的是学习嵌入式 Linux 系统。随着我们课程的陆续发布，我们也会给 QEMU 添加更多的功能。比如后面要讲到触摸屏时，就会修改 QEMU 让它支持触摸屏。

所以你需要不断地更新 QEMU，有 3 个方法。

4.1 下载最新的 release 包

从下面 2 个地址就可以下载到最新的 release 包，解压开后即可使用：

- ① ubuntu-18.04 开发环境下 qemu imx6ul 系统镜像下载地址：

http://wiki.100ask.org/Download_ubuntu-18.04_imx6ul_qemu

- ② ubuntu-16.04 开发环境下 qemu imx6ul 系统镜像下载地址：

http://wiki.100ask.org/Download_ubuntu-16.04_imx6ul_qemu

注意：如果你之前对内核、设备树或是文件系统进行过修改，想继续使用这些修改过的文件的话，需要用它们来覆盖 release 包中解压出来的对应文件。

4.2 下载最新的 qemu-system-arm 可执行程序及配置文件

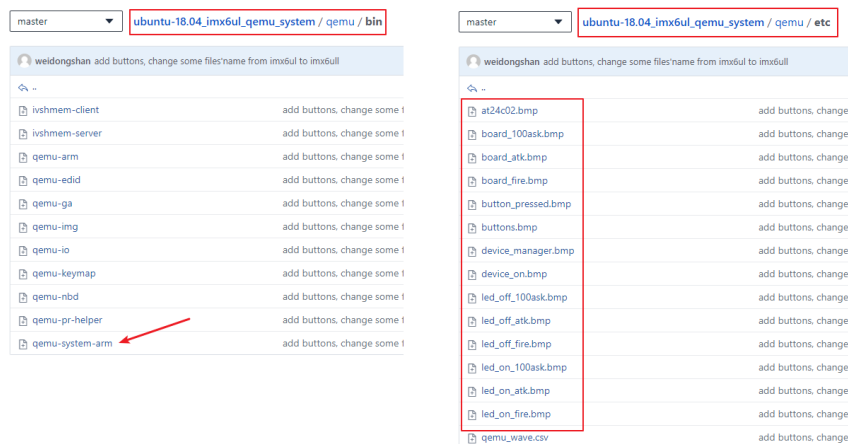
如果你不想下载整个 release 包，可以只替换其中的某些文件。

可以打开下面的某个网址(根据你的 ubuntu 版本号选择)：

https://dev.tencent.com/u/weidongshan/p/ubuntu-16.04_imx6ul_qemu_system/git

https://dev.tencent.com/u/weidongshan/p/ubuntu-18.04_imx6ul_qemu_system/git

以 ubuntu 18.04 为例，进入下图所示的 bin、etc 目录：



对于 bin 目录，只需要下载其中的 qemu-system-arm；对于 etc 目录，里面所有的 bmp 文件都要下载。

你之前在 Ubuntu 中曾经安排过我们提供的 QEMU，你用下载的文件覆盖 QEMU 中的对应文件即可。

4.3 下载最新的 qemu 源码并编译

请参考后续章节，这属于对 QEMU 的开发了。对于嵌入式 Linux 初学者，没必要研究 QEMU，使用我们提供的可执行程序即可。

B. QEMU 开发手册

重要的事情说三遍：

对于嵌入式 Linux 初学者，对于 QEMU 无感者，对于急于找工作的人，
不需要深入研究 QEMU，不需要看本开发手册，看前面的使用手册就可以了，会用就行。
不需要深入研究 QEMU，不需要看本开发手册，看前面的使用手册就可以了，会用就行。
不需要深入研究 QEMU，不需要看本开发手册，看前面的使用手册就可以了，会用就行。

我们使用 QEMU 来模拟 IMX6ULL 开发板，就像去制作一个开发板一样。作为初学者，你并不需要去设计开发板，会用开发板就可以了。

如果你想深入研究 QEMU，想承接我们发布的 QEMU 外包项目，那么请阅读本手册。

QEMU 框架简单，功能却很强大。

理解了它的框架之后，添加新的外设并不困难。当然这需要你对外设的原理有清楚的认识。

我们开发 QEMU，要做的事情主要有 3 部分：

1. 添加外设

比如添加一个 LED，那么在 QEMU 中得有对应的源码，这些源码要监测 LED 对应的寄存器的值：当 APP 写寄存器时，这些源码要把那些值记录下来。

2. 添加外设的 GUI

QEMU 的界面很简陋，它默认只实现了 LCD 的 GUI 显示。你想点亮 LED，在 QEMU 上是看不出效果的。

所以我们需要给外设添加 GUI 界面。

比如上面举的 LED 例子，APP 写寄存器时，除了把寄存器的值记录下来之后，我们还要在 GUI 界面显示一个 LED，并且把它点亮或熄灭。

3. 在虚拟开发板上开发测试程序

这个测试程序可以是裸机，也可以是 Linux 驱动+应用程序。

我们在 QEMU 模拟出来的板子上运行这些测试程序时，跟真实板子应该没有差别。

上述列的第 1、2 部分，需要对 QEMU 有所了解，请看本文的“开发手册”。

上述列的第 3 部分，跟 QEMU 无关，跟在真实板子上写的程序是一样的。

1. QEMU 下载与编译

1.1 源码下载

官方的 QEMU 对 IMX6ULL 的支持还太弱，没有更形象化的 GUI，也没有更多的外设。我们对它进行了大量的改进。

修改后的源码位于这 2 个 GIT 中：

<https://github.com/100askTeam/qemu.git>

<https://gitee.com/weidongshan/qemu.git>

在 Ubuntu 下，执行如下命令即可下载：

```
$ git clone https://gitee.com/weidongshan/qemu.git
或
$ git clone https://github.com/100askTeam/qemu.git
```

我们会不断更新 QEMU，你下载过 QEMU 之后，可以进入 qemu 目录执行下述命令更新代码：

```
$ git pull origin
```

1.2 配置、编译、安装

进入 qemu 目录，执行如下配置命令：

```
./configure --prefix=$PWD/ --target-list="arm-softmmu arm-linux-user" --enable-debug
--enable-sdl --enable-kvm --enable-tools --disable-curl
```

编译、安装命令如下：

```
$ make
$ make install
```

如果一切正常，会在 qemu 源码目录下生成 bin 子目录，里面存放有各种可执行程序。

配置、编译过程中有可能出错，一般就是缺少库文件。

如果你的 ubuntu 能上网，那么使用 apt-get 命令即可安装这些库。

错误示例，提示信息如下：

```
ERROR: pixman >= 0.21.8 not present.
Please install the pixman devel package.
```

解决方法：

① 确定库的名称：

执行如下命令：

```
$ apt-cache search pixman
libpixman-1-0 - pixel-manipulation library for X and cairo
libpixman-1-dev - pixel-manipulation library for X and cairo (development files)
```

根据输出信息，需要安装开发包(dev 表示开发包): libpixman-1-dev。

② 安装开发包:

```
$ sudo apt-get install libpixmap-1-dev
```

可能你的 ubuntu 中已经安装了某些开发包, 下面列出一些必须的包:

```
$ sudo apt-get install pkg-config
$ sudo apt-get install libSDL2-dev
$ sudo apt-get install libpixmap-1-dev
```

每次出错后, 根据提示信息安装开发包, 然后重新配置、编译、安装。

如果一切正常, 在当前目录下会生成 bin 子目录, 里面有生成的 QEMU 程序:

```
qemu-system-arm
```

1.3 使用新的 qemu-system-arm

将上面编译出来的 bin/qemu-system-arm 可执行文件复制到如下目录:

```
ubuntu-18.04_imx6ul_qemu_system/qemu/bin
或
ubuntu-16.04_imx6ul_qemu_system/qemu/bin
```

我们可能添加了更多的 GUI 显示, 这些 GUI 所用图片位于源码目录的 etc 子目录下, 这些图片也需要复制到如下目录去:

```
ubuntu-18.04_imx6ul_qemu_system/qemu/etc
或
ubuntu-16.04_imx6ul_qemu_system/qemu/etc
```

然后就可以执行 qemu-imx6ull-gui.sh 或 qemu-imx6ull-nogui.sh 来使用您编译出来的 QEMU 了。

1.4 QEMU 源码目录

我们只罗列出涉及的少许文件, 一般来说一个.c 文件会有一个.h 文件, 它们的目录类似。比如 hw/gpio/imx_gpio.c 对应的头文件为 include/hw/gpio/imx_gpio.h

```

├── accel
│   ├── kvm
│   ├── stubs
│   └── tcg
├── etc
├── hw
│   ├── arm          // 硬件相关
│   ├── cpu          // arm相关
│   ├── display      // CPU相关
│   ├── gpio         // 显示相关
│   ├── i2c          // gpio相关
│   └── i2c          // i2c相关
├── include           // 头文件
│   ├── hw           // 硬件相关
│   │   ├── arm      // arm相关
│   │   ├── cpu      // CPU相关
│   │   ├── display  // 显示相关
│   │   ├── gpio     // gpio相关
│   │   └── i2c      // i2c相关
│   └── ui
└── ui
```

2. QEMU 的设备创建过程

2.1 重要结构体 TypeInfo

一个板子上有很多硬件：IMX6ULL、LED、按键、LCD、触摸屏、网卡等等。

IMX6ULL 这类芯片被称为 SoC(System on Chip), 它里面也有很多部件, 比如 CPU、GPIO、SD 控制器、中断控制器等等。

这些硬件, 或是部件, 各有不同。怎么描述它们?

每一个都使用一个 TypeInfo 结构体来描述。

比如对于 CPU, 有这样的结构体(hw/cpu/a15mpcore.c):

```
static const TypeInfo a15mp_priv_info = {
    .name = TYPE_A15MPCORE_PRIV,
    .parent = TYPE_SYS_BUS_DEVICE,
    .instance_size = sizeof(A15MPPrivState),
    .instance_init = a15mp_priv_initfn,
    .class_init = a15mp_priv_class_init,
};
```

对于 LCD, 有这样的结构体(hw/display/100ask_qemu_fb.c):

```
static const TypeInfo ask100fb_info = {
    .name = TYPE_100ASKFB,
    .parent = TYPE_SYS_BUS_DEVICE,
    .instance_size = sizeof(ASK100FbState),
    .class_init = ask100fb_class_init,
    .instance_init = ask100fb_init,
};
```

对于 GPIO, 也有这样的结构体(hw/gpio/imx_gpio.c):

```
static const TypeInfo imx_gpio_info = {
    .name = TYPE_IMX_GPIO,
    .parent = TYPE_SYS_BUS_DEVICE,
    .instance_size = sizeof(IMXGPIOState),
    .class_init = imx_gpio_class_init,
};
```

由此可见, 在 QEMU 中, 每一个硬件都由一个 TypeInfo 来描述。这些结构体都会被注册进程序里, 在一个链表中保存着, 备用。注意, 是备用, 它们并不一定会用到。

怎么注册这些 TypeInfo 结构体呢？不需要我们去调用注册函数，以 GPIO 为例，在 hw/gpio/imx_gpio.c 中有如下代码：

```
static void imx_gpio_register_types(void)
{
    type_register_static(&imx_gpio_info);
}

type_init(imx_gpio_register_types)
```

关键点在于 type_init，这个宏在 include/qemu/module.h 中定义：

```
#define type_init(function) module_init(function, MODULE_INIT_QOM)

#define module_init(function, type) \
static void __attribute__((constructor)) do_qemu_init_##function(void) \
{ \
    register_module_init(function, type); \
}

void register_module_init(void (*fn)(void), module_init_type type)
{
    ModuleEntry *e;
    ModuleTypeList *l;

    e = g_malloc0(sizeof(*e));
    e->init = fn;
    e->type = type;

    l = find_type(type);
    QTAILQ_INSERT_TAIL(l, e, node);
}
```

对于属性为“constructor”函数，它在 main 函数之前被调用。

对于上述的 type_init(imx_gpio_register_types)，它的宏展开如下：

```
static void __attribute__((constructor)) do_qemu_init_imx_gpio_register_types(void) \
{ \
    register_module_init(imx_gpio_register_types, MODULE_INIT_QOM); \
}
```

这就得到了一个属性为“constructor”的函数 do_qemu_init_imx_gpio_register_types，这个函数将在 main 函数之前被调用，它调用了 register_module_init。

register_module_init 中构造了一个 ModuleEntry 结构体，并把它添加进某个链表里，这个链表是 init_type_list[MODULE_INIT_QOM]：

```
void register_module_init(void (*fn)(void), module_init_type type)
{
    ModuleEntry *e;
    ModuleTypeList *l;

    e = g_malloc0(sizeof(*e));
    e->init = fn; // 等于imx_gpio_register_types
    e->type = type;

    l = find_type(type);
    QTAILQ_INSERT_TAIL(l, e, node);
}
```

这样就得到了一个链表：

init_type_list[MODULE_INIT_QOM]



2.2 生成 TypeImpl

从名字上讲, TypeImpl 就是 TypeInfo 的实现, 就是使用 TypeInfo 的信息构造一个 TypeImpl 结构体。

在 init_type_list[MODULE_INIT_QOM]链表中, 有一系列的 ModuleEntry 结构体, 每个 ModuleEntry 结构体中有一个 init 函数, 它指向某个 xxx_register_types, 比如:

```
a15mp_register_types
ask100fb_register_types
```

这些 ModuleEntry 结构体中的 init 函数何时被调用?

vl.c:

```
main
module_call_init(MODULE_INIT_QOM);
    QTAILQ_FOREACH(e, l, node) {
        e->init();
    }
```

这些 xxx_register_types 执行后, 又得到了什么?

① 分配一个 TypeImpl 结构体, 使用 TypeInfo 来设置它:

```
static TypeImpl *type_new(const TypeInfo *info)
{
    TypeImpl *ti = g_malloc0(sizeof(*ti));
    int i;

    g_assert(info->name != NULL);

    if (type_table_lookup(info->name) != NULL) {
        fprintf(stderr, "Registering '%s' which already\n", info->name);
        abort();
    }

    ti->name = g_strdup(info->name);
    ti->parent = g_strdup(info->parent);

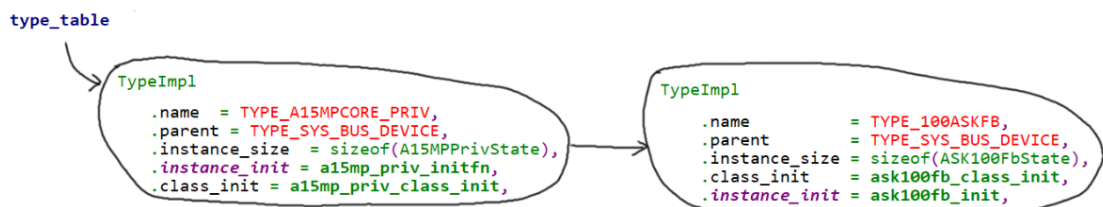
    ti->class_size = info->class_size;
    ti->instance_size = info->instance_size;

    ti->class_init = info->class_init;
    ti->class_base_init = info->class_base_init;
    ti->class_data = info->class_data;

    ti->instance_init = info->instance_init;
```

② 把 TypeImpl 放入链表: type_table

于是, 在程序中就有了这样的链表:



2.3 使用 TypeImpl: 实例化

在程序的 type_table 链表中，有很多 TypeImpl 结构体，比如 CPU、GPIO、LED、LCD 对应的 TypeImpl 结构体。

但是这并不表示 QEMU 模拟的板子上有这些硬件，毕竟它们只是“TypeImpl”，表示“类型”，需要在“实例化”之后，才表示板子上有了这些硬件。

以 CPU 为例，代码为 hw/cpu/a15mpcore.c，里面声明了一个 A15MPPrivState 结构体，还定义了一个 TypeInfo 结构体：

```
typedef struct A15MPPrivState {  
    /*< private >*/  
    SysBusDevice parent_obj;  
    /*< public >*/  
  
    uint32_t num_cpu;  
    uint32_t num_irq;  
    MemoryRegion container;  
  
    GICState gic;  
} A15MPPrivState;  
  
static const TypeInfo a15mp_priv_info = {  
    .name = TYPE_A15MPCORE_PRIV,  
    .parent = TYPE_SYS_BUS_DEVICE,  
    .instance_size = sizeof(A15MPPrivState),  
    .instance_init = a15mp_priv_initfn,  
    .class_init = a15mp_priv_class_init,  
};
```

A15MPPrivState 和 TypeInfo、TypeImpl 之间有什么关系？

TypeImpl 的信息基本来自 TypeInfo，

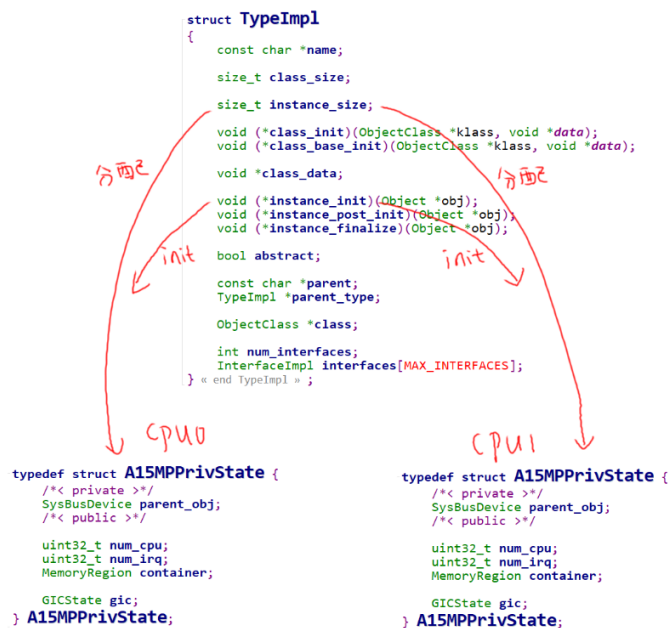
所以问题转为：A15MPPrivState 和 TypeInfo 之间有什么关系？

A15MPPrivState 用来表示一个 CPU，你要在板子上添加一个 CPU，必须分配、设置一个 A15MPPrivState 结构体。

板子上的主芯片可能是单核 CPU 的，也可能是多核 CPU 的。

假设有 2 个 CPU，那么应该有对应的 2 个 A15MPPrivState 结构体。这 2 个 CPU 是类似的，同属于某类：用 TypeImpl 来描述。

所以，可以得到下面的图：



谁来分配、设置 A15MPPrivState 结构体？

① 分配：

猜测是根据 TypeInfo 中的 instance_size 来 malloc 出 A15MPPrivState 结构体。

② 设置：

猜测是调用 TypeInfo 中的 instance_init 函数来设置刚 malloc 出 A15MPPrivState 结构体。

谁来 malloc、谁来调用 TypeInfo 中的 instance_init 函数？

有 2 种方法：

① qdev_create/qdev_init_nofail

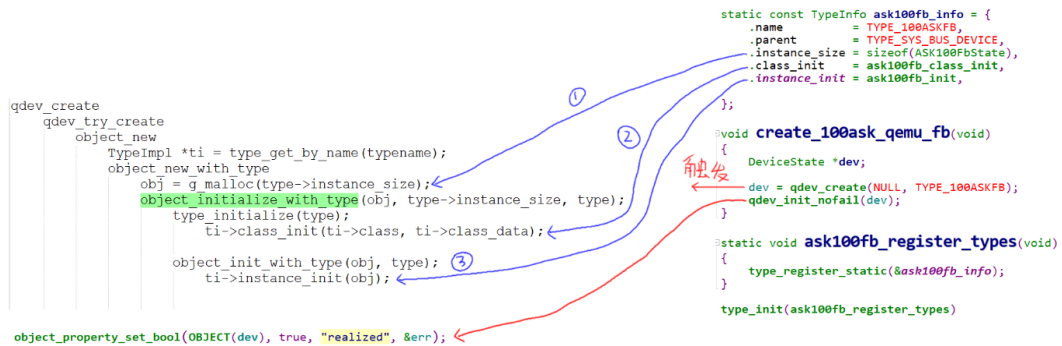
② sysbus_init_child_obj/object_property_set_bool

2.4 实例化方法 1: qdev_create/qdev_init_nofail

这 2 个函数是成对出现的，以 hw/display/100ask_qemu_fb.c 为例：

```
void create_100ask_qemu_fb(void)
{
    DeviceState *dev; 成对出现
    dev = qdev_create(NULL, TYPE_100ASKFB);
    qdev_init_nofail(dev);
}
```

这 2 个函数的作用如下图所示：



2.4.1 qdev_create 分析

- ① 第 2 个参数是 name，会被用来找到对应的 TypeInfo 结构体
- ② 分配 instance_size 大小的内存，即分配 ASK100FbState 结构体，这用来表示 LCD
- ③ 调用 TypeInfo 结构体中的 class_init 函数

class_init，顾名思义，这个设备属于什么类别？先初始化一下它的类别。

这些 class_init 函数都很类似，都是设置 dc->realize 函数，比如：

```
static void a15mp_priv_class_init(ObjectClass *klass, void *data)
{
    DeviceClass *dc = DEVICE_CLASS(klass);
    dc->realize = a15mp_priv_realize;
    dc->props = a15mp_priv_properties;
    /* We currently have no savable state */
}

static void ask100fb_class_init(ObjectClass *oc, void *data)
{
    DeviceClass *dc = DEVICE_CLASS(oc);
    set_bit(DEVICE_CATEGORY_DISPLAY, dc->categories);
    dc->realize = ask100fb_realize; /* ask100fb_realize 何时被调用 */
    /* Note: This device does not any state that we have to reset or migrate */
}
```

- ④ 调用 TypeInfo 结构体中的 instance_init 函数

2.4.2 qdev_init_nofail 分析

qdev_init_nofail 做的事情很简单：

```
void qdev_init_nofail(DeviceState *dev)
{
    Error *err = NULL;

    assert(!dev->realized);

    object_ref(OBJECT(dev));
    object_property_set_bool(OBJECT(dev), true, "realized", &err);
    if (err) {
        error_reportf_err(err, "Initialization of device %s failed: ",
                          object_get_typename(OBJECT(dev)));
        exit(1);
    }
    object_unref(OBJECT(dev));
}
```

属性值 属性名

只是把设备的 realized 属性设置为 true，表示可以对它进行 realize(变为现实)了。这会导致 dc->realize 函数被调用，即设备的类里的 realize 函数被调用。

2.4.3 总结：怎么创建设备

怎么定义一个设备？如下图：

```
static const TypeInfo ask100fb_info = {1.
    .name = TYPE_100ASKFB,
    .parent = TYPE_SYS_BUS_DEVICE,
    .instance_size = sizeof(ASK100FbState), /* 0
    .class_init = ask100fb_class_init,
    .instance_init = ask100fb_init, /* 0
};

void create_100ask_qemu_fb(void)
{
    DeviceState *dev;

    dev = qdev_create(NULL, TYPE_100ASKFB); 4.
    qdev_init_nofail(dev); 5.
}

static void ask100fb_register_types(void) 2.
{
    type_register_static(&ask100fb_info);
}

type_init(ask100fb_register_types) 3.
```

① 先定义一个 TypeInfo 结构体

里面有 name，表示它的类型名。

有 class_init，这是“类别的初始化函数”，该类下可能有多个设备。class_init 函数中通常给该类设置一个 realize 函数。

有 instance_size，每一个设备都用一个结构体来表示，比如 LCD 用 ASK100FbState 来描述。Instance_size 表示这个结构体的大小。

有 instance_init，这是“实例的初始化函数”，它会被用来初始化设备结构体，比如初始化 ASK100FbState 结构体。

② 注册这个 TypeInfo 结构体：

定义一个 ask100fb_register_types 函数，里面会注册 TypeInfo 结构体。

- ③ 使用 type_init，把 ask100fb_register_types 函数放入链表中
- ④ 调用 qdev_create 创建设备，这会传入 type name
- ⑤ 调用 qdev_init_nofail 设置设备的状态为 realized，这会导致类别的 realize 函数被调用。

简单地说，一个设备被创建时，这些函数被依次调用：

- ① TypeInfo 中的 class_init：它会设置 dc->realize = 某个函数
- ② TypeInfo 中 instance_size 大小的对象被 malloc
- ③ TypeInfo 中的 instance_init 函数被调用，它被用来初始化步骤②中 malloc 出来的结构体
- ④ dc->realize 被调用

2.5 实例化方法 2：object_initialize_child/object_property_set_bool

参考 hw/arm/fsl-imx6ul.c，里面大量的成对代码，比如：

```
object_initialize_child(obj, "cpu0", &s->cpu, sizeof(s->cpu),
                        ARM_CPU_TYPE_NAME("cortex-a7"), &error_abort, NULL);

object_property_set_bool(OBJECT(&s->cpu), true,
                          "realized", &error_abort);
```

上述函数的内部调用过程，跟 qdev_create/qdev_init_nofail 是类似的：



2.5.1 object_initialize_child 分析

该函数的第 5 个参数是 type，表示 type name，它会被用来找到对应的 TypeInfoImpl。
找到后，会分配 instance_size 大小的结构体；
然后调用 TypeInfoImpl 中的 class_init 函数，这一般是设置 dc->realize。
最后调用 TypeInfoImpl 中的 instance_init 函数。

2.5.2 object_property_set_bool 分析

比如：

```
object_property_set_bool(OBJECT(&s->cpu), true,
                          "realized", &error_abort);
```

只是把设备的 realized 属性设置为 true，表示可以对它进行 realize(变为现实)了。
这会导致 dc->realize 函数被调用，即设备的类里的 realize 函数被调用。

2.5.3 总结：怎么创建设备

怎么定义一个设备？如下图所示：

```
static void a15mp_priv_class_init(ObjectClass *klass, void *data)
{
    DeviceClass *dc = DEVICE_CLASS(klass);

    dc->realize = a15mp_priv_realize;
    dc->props = a15mp_priv_properties;
    /* We currently have no savable state */
}

static const TypeInfo a15mp_priv_info = {1.
    .name = TYPE_A15MPCORE_PRIV,
    .parent = TYPE_SYS_BUS_DEVICE,
    .instance_size = sizeof(A15MPPrivState),
    .instance_init = a15mp_priv_initfn,
    .class_init = a15mp_priv_class_init,
};

static void a15mp_register_types(void) 2.
{
    type_register_static(&a15mp_priv_info);
}

type_init(a15mp_register_types) 3.
```

① 先定义一个 TypeInfo 结构体

里面有 name，表示它的类型名。

有 class_init，这是“类别的初始化函数”，该类下可能有多个设备。class_init 函数中通常给该类设置一个 realize 函数。

有 instance_size，每一个设备都用一个结构体来表示，比如 CPU 用 A15MPPrivState 来描述。Instance_size 表示这个结构体的大小。

有 instance_init，这是“实例的初始化函数”，它会被用来初始化设备结构体，比如初始化 A15MPPrivState 结构体。

② 注册这个 TypeInfo 结构体：

定义一个 a15mp_register_types 函数，里面会注册 TypeInfo 结构体。

③ 使用 type_init，把 a15mp_register_types 函数放入链表中

④ 调用 object_initialize_child 创建设备，这会传入 type name

⑤ 调用 object_property_set_bool 设置设备的状态为 realized，这会导致类别的 realize 函数被调用。

2.6 怎么创建设备

其实在 2.4.3、2.5.3 已经整理出来了，在这里只是再次总结一下。

2.6.1 设置 TypeInfo 结构体

比如 LCD：

```
static const TypeInfo ask100fb_info = {
    .name          = TYPE_100ASKFB,
    .parent        = TYPE_SYS_BUS_DEVICE,
    .instance_size = sizeof(ASK100FbState),
    .class_init    = ask100fb_class_init,
    .instance_init = ask100fb_init,
};
```

要注意到，里面有一个 ASK100FbState 结构体，这个结构体由我们自己设置。但是它的格式有一定要求：

```
struct ASK100FbState {
    SysBusDevice parent_obj; /* 这个必须有,并且必须在最前面 */
    MemoryRegion iomem;
    MemoryRegionSection fbsection;
    QemuConsole *con;

    unsigned int fb_base_phys;
    unsigned int fb_xres;
    unsigned int fb_yres;
    unsigned int fb_bpp;
    unsigned int board_xres;
    unsigned int board_yres;
    int invalidate;
};
```

2.6.2 注册 TypeInfo 结构体

比如：

```
static void ask100fb_register_types(void)
{
    type_register_static(&ask100fb_info);
}

type_init(ask100fb_register_types)
```

2.6.3 使用 TypeInfo 创建设备/设置设备的 realized 属性为 true

这有 2 种方法，前面介绍过：

```
qdev_create/qdev_init_nofail
object_initialize_child/object_property_set_bool
```

3. QEMU 的设备模拟

3.1 QEMU 模拟外设的原理

QEMU 主要是实现了 CPU 核的模拟，可以读写某个地址。

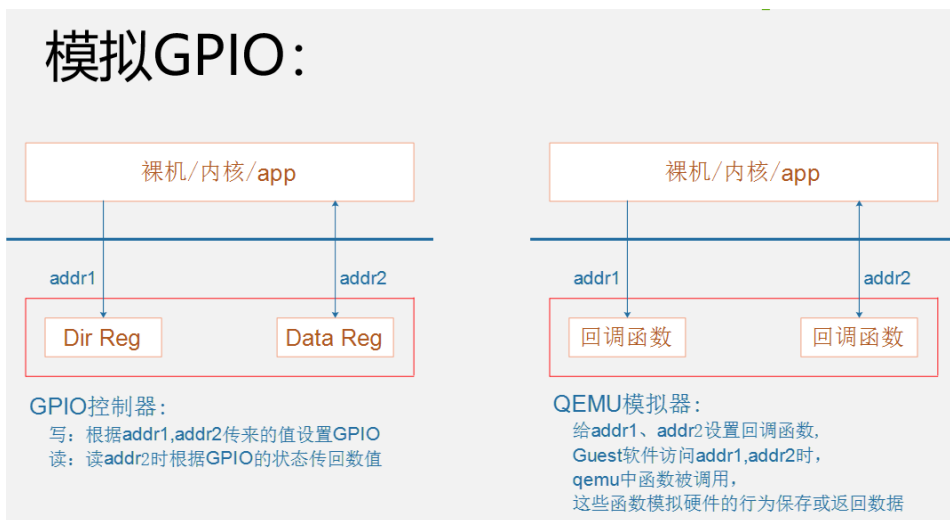
QEMU 的模拟外设的原理很简单：**硬件即内存**。

要在 QEMU 上模拟某个外设，思路就是：

- ① CPU 读某个地址时，QEMU 模拟外设的行为，把数据返回给 CPU
- ② CPU 写某个地址时，QEMU 获得数据，用来模拟外设的行为。

即：要模拟外设备，我们只需要针对外设的地址提供对应的读写函数即可。

以 GPIO 为例：



以 LCD 控制器为例，它主要有 2 大功能：

- ① 写 LCD 控制器，根据外接的 LCD 设置参数，比如分辨率、BPP、各种时序
- ② 从 FrameBuffer 中不断获得数据发给 LCD，在 LCD 上显示出来。

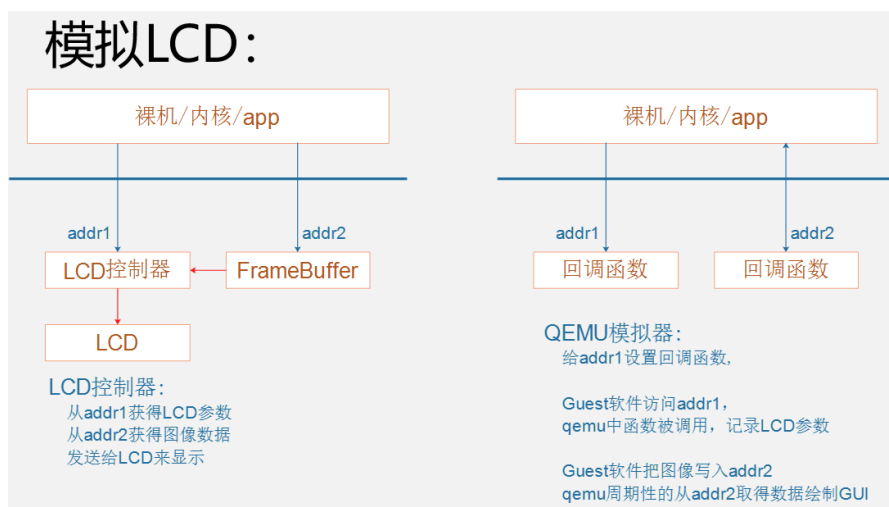
站在 Linux LCD 驱动的角度，上述 2 大功能可以分为：

- ① 写 LCD 控制器的相关寄存器
- ② 分配 FrameBuffer，写 FrameBuffer

QEMU 中要模拟 LCD 控制器，需要：

- ① 记录驱动程序写寄存器的值，解析出分辨率等信息
- ② 记录 FrameBuffer 的地址，并持续不断地从中得到图像数据并显示出来

框图如下：



简单地说,

① 设置 LCD 控制器时:

在 QEMU 中可以给 LCD 控制器的访问地址 A1 ~ A2 提供读写回调函数, 比如:

```
qemu_a1a2_read
qemu_a1a2_write
```

当 Linux LCD 驱动程序写 LCD 控制器的寄存器时, 就会导致 qemu 中的 qemu_a1a2_write 函数被调用, 在函数中分析、记录这些值, 得到分辨率等信息。

② 写 FrameBuffer 时:

在 QEMU 中针对 FrameBuffer 提供一个刷新函数。

当 LCD 驱动写 FrameBuffer 时, QEMU 会使用这些数据更新 GUI 窗口的图像。

3.2 给某段地址提供读写函数

怎么描述某段地址：基地址、大小；还得给这段地址提供读写函数。
这段地址设置好后，需要添加进 system_memory 去。
有 2 种方法。

3.2.1 memory_region_init_io/memory_region_add_subregion

比如：

```
static void ask100fb_init(Object *obj)
{
    ASK100FbState *s = ASK100FB(obj);
    MemoryRegion *address_space = get_system_memory();

    memory_region_init_io(&s->iomem, obj, &ask100_lcd_ops, s, TYPE_100ASKFB, 16);
    memory_region_add_subregion(address_space, FSL_IMX6UL_LCDIF_ADDR, &s->iomem);
}
```

1. 初始化 iomem：读写函数、大小

2. 把 iomem 添加进 system_mem：并指定基地址

上图中 memory_region_init_io 被用来初始化一块内存 s->iomem，指定了它的读写函数、大小。

然后给 s->iomem 指定了基地址，并添加进 system_memory 中。

以后，客户机上的程序读写这块地址时，就会导致对应的读写函数被调用。

3.2.2 memory_region_init_io/sysbus_init_mmio/sysbus_mmio_map

以 hw/net/smc91c111.c 为例：

```
static void smc91c111_realize(DeviceState *dev, Error **errp)
{
    SysBusDevice *sbd = SYS_BUS_DEVICE(dev);
    smc91c111_state *s = SMC91C111(dev);

    memory_region_init_io(&s->mmio, OBJECT(s), &smc91c111_mem_ops, s,
                        "smc91c111-mmio", 16);
    sysbus_init_mmio(sbd, &s->mmio);
    sysbus_init_irq(sbd, &s->irq);
    qemu_macaddr_default_if_unset(&s->conf.macaddr);
    s->nic = qemu_new_nic(&net_smc91c111_info, &s->conf,
                        object_get_typename(OBJECT(dev)), dev->id, s);
    qemu_format_nic_info_str(qemu_get_queue(s->nic), s->conf.macaddr.a);
    /* ??? Save/restore. */
}

/* Legacy helper function. Should go away when machine config files are
   implemented. */
void smc91c111_init(NICInfo *nd, uint32_t base, qemu_irq irq)
{
    DeviceState *dev;
    SysBusDevice *s;

    qemu_check_nic_model(nd, "smc91c111");
    dev = qdev_create(NULL, TYPE_SMC91C111);
    qdev_set_nic_properties(dev, nd);
    qdev_init_nofail(dev);
    s = SYS_BUS_DEVICE(dev);
    sysbus_mmio_map(s, 0, base);
}
```

1. 初始化 mmio：指定读写函数、大小

2. 把 mmio 传给设备

3. 从设备中把 mmio 添加进 system_memory 并指定基地址

4. 模拟 LED

4.1 主要函数

客户机上的程序要操作 LED 时，会先设置 GPIO 为 output，然后把值写入某个数据寄存器。

所以我们只需要针对数据寄存器提供对应的 write 函数即可

QEMU 中已经实现 IMX6ULL 的 GPIO 模拟，代码在 hw/gpio/imx_gpio.c 中。

对应的 write 函数如下：

```
static void imx_gpio_write(void *opaque, hwaddr offset, uint64_t value,
                           unsigned size)
{
    IMXGPIOState *s = IMX_GPIO(opaque);

    DPRINTF("(%s, value = 0x%" PRIx32 ")\\n", imx_gpio_reg_name(offset),
            (uint32_t)value);

    switch (offset) {
        case DR_ADDR:
            s->dr = value;
            imx_gpio_set_all_output_lines(s);
            imx_gpio_info_update(opaque, offset, value, size);
            break;
    }
```

我们添加的代码，它根据值来在GUI中显示LED

4.2 添加一段内存

客户机的程序要设置 GPIO5_IO03 为输出引脚时，需要访问这个寄存器：

IOMUXC_SNVS_SW_MUX_CTL_PAD_SNVS_TAMPER3

它的基地址是 0x2290014，QEMU 中并没有添加这个地址，客户机访问它时就会发生硬件错误。

所以在 hw/arm/fsl-imx6ul.c 中使用如下代码添加了这块空间：

```
/*
 * IOMUXC_SNVS memory, 100ask
 */
memory_region_init_rom(&s->iomuxc_snvs, NULL, "imx6ul.iomux_snvs",
                      FSL_IMX6UL_IOMUXC_SNVS_SIZE, &error_abort);
memory_region_add_subregion(get_system_memory(), FSL_IMX6UL_IOMUXC_SNVS,
                             &s->iomuxc_snvs);
```

5. 模拟 LCD

我们添加的文件是：hw/display/100ask_qemu_fb.c，新加一个文件时要把它编进 QEMU 中，需要修改同目录的 Makefile.objs，比如修改 hw/display/Makefile.objs，添加一行：

```
common-obj-$(CONFIG_FSL_IMX6UL) += 100ask_qemu_fb.o
```

5.1 在 Linux 上编写 LCD 驱动程序

既然操作的不是真实的 LCD 控制器，那么 LCD 驱动程序可以极大精简。

- ① 对于 LCD 控制器，只需要操作 4 个寄存器：

分别用来保存：framebuffer 的物理地址、宽度、高度、BPP。

你需要记住这些寄存器的物理地址(可以自己指定地址是什么)。

- ② 对于 FrameBuffer：

驱动程序分配得到 FrameBuffer 后，要把它物理地址写到上述第 1 个寄存器里。

部分代码如下，其他的时钟使能、GPIO 设置等等都不再需要：

```
/* 3. 硬件相关的操作 */
/* 3.1 配置GPIO用于LCD */

/* 3.2 根据LCD手册设置LCD控制器，比如VCLK的频率等 */
lcd_regs = ioremap(FSL_IMX6UL_LCDIF_ADDR, sizeof(struct lcd_regs));

/* 3.3 分配显存(framebuffer)，并把地址告诉LCD控制器 */
s3c_lcd->screen_base = dma_alloc_writecombine(NULL, s3c_lcd->fix.smem_len, (dma_addr_t *)&s3c_lcd->fix.smem_start,

/* 设置LCD控制器 */
printf("%s %s %d\n", __FILE__, __FUNCTION__, __LINE__);
lcd_regs->fb_base_phys = s3c_lcd->fix.smem_start;
lcd_regs->fb_xres = LCD_XRES;
lcd_regs->fb_yres = LCD_YRES;
lcd_regs->fb_bpp = LCD_BPP;
```

5.2 在 QEMU 中创建 LCD 控制器的设备

```
static const TypeInfo ask100fb_info = {
    .name = TYPE_100ASKFB,
    .parent = TYPE_SYS_BUS_DEVICE,
    .instance_size = sizeof(ASK100FbState), /* object_new_with_type 创建设备时会malloc该结构体，并传给instance_init */
    .class_init = ask100fb_class_init, /* object_init_with_type */
    .instance_init = ask100fb_init,
};

void create_100ask_qemu_fb(void)
{
    DeviceState *dev;

    dev = qdev_create(NULL, TYPE_100ASKFB);
    qdev_init_nofail(dev);
}

static void ask100fb_register_types(void)
{
    type_register_static(&ask100fb_info);
}

type_init(ask100fb_register_types)
```

4. 创建设备时，会分配ASK100FbState结构体
5. 初始化实例
6. 初始化类别
2. 调用此函数
3. 创建qdev，传入名字
1. 先定义qdev，里面有名字

5.3 重要函数分析

5.3.1 给 LCD 控制器的寄存器提供回调函数

```
static void ask100fb_init(Object *obj)
{
    ASK100FbState *s = ASK100FB(obj); QEMU提供读写回调函数
    MemoryRegion *address_space = get_system_memory();
    memory_region_init_io(&s->iomem, obj, &ask100_lcd_ops, s, TYPE_100ASKFB, 16);
    memory_region_add_subregion(address_space, FSL_IMX6UL_LCDIF_ADDR, &s->iomem);
}

static void ask100_lcd_write(void *opaque, hwaddr addr,
                             uint64_t value, unsigned size)
{
    ASK100FbState *s = ASK100FB(opaque);

    switch (addr) {
        case 0x00:
            s->fb_base_phys = value; 导致此函数被调用
            break;

        case 0x04:
            s->fb_xres = value;
            qemu_console_resize(s->con, s->fb_xres, s->fb_yres);
            break;

        case 0x08:
            s->fb_yres = value;
            qemu_console_resize(s->con, s->fb_xres, s->fb_yres);
            break;

        case 0x0c:
            s->fb_bpp = value;
            break;
        default:
            break;
    } « end switch addr »
} « end ask100_lcd_write »

static const MemoryRegionOps ask100_lcd_ops = {
    .read = ask100_lcd_read,
    .write = ask100_lcd_write,
    .endianness = DEVICE_NATIVE_ENDIAN,
};
```

LCD控制器基地址、大小

Linux LCD驱动写LCD控制器时，记录其中的信息

5.3.2 给 FrameBuffer 提供更新函数

```
static void ask100fb_invalidate(void *opaque)
{
    ASK100FbState *s = ASK100FB(opaque);
    s->invalidate = 1;
}

static const GraphicHwOps ask100fb_ops = {
    .invalidate = ask100fb_invalidate,
    .gfx_update = ask100fb_update,
};

static void ask100fb_realize(DeviceState *dev, Error **errp)
{
    ASK100FbState *s = ASK100FB(dev);

    s->invalidate = 1;
    s->fb_xres = 800;
    s->fb_yres = 600;
    s->con = graphic_console_init(dev, 0, &ask100fb_ops, s);
    qemu_console_resize(s->con, s->fb_xres, s->fb_yres);
}
```

给graphic提供更新函数

5.3.3 更新函数

```
static void ask100fb_update(void *opaque)
{
    ASK100FbState *s = ASK100FB(opaque);
    SysBusDevice *sbd;
    DisplaySurface *surface = qemu_console_surface(s->con);

    int dest_width;
    int src_width;
    int first = 0;
    int last = 0;

    src_width = s->fb_xres * s->fb_bpp / 8;
    dest_width = s->fb_xres * surface_bits_per_pixel(surface) / 8;

    sbd = SYS_BUS_DEVICE(s);
    if (s->invalidate) {
        framebuffer_update_memory_section(&s->fbsection, sysbus_address_space(sbd), s->fb_base_phys,
                                          s->fb_yres, src_width);
    }

    framebuffer_update_display(surface, &s->fbsection, s->fb_xres, s->fb_yres,
                               src_width, dest_width, 0, s->invalidate, ask100fb_draw_line_src16,
                               s, &first, &last);

    if (first >= 0) {
        dpy_gfx_update(s->con, 0, first, s->fb_xres, last - first + 1);
    }
    s->invalidate = 0;
} « end ask100fb_update »
```

有待改进

FrameBuffer物理基地址

确定要根据那块内存来更新现实？

更新显示

6. QEMU 的输出：GUI 系统

QEMU 的 GUI 系统，支持 SDL、GTK 等。SDL 使用比较简单，我们就使用 SDL 来显示 GUI。

SDL 的显示原理跟 LCD FrameBuffer 是一样的，可以认为每一个 SDL 窗口都有一个显存。你可以在显存中修改每一个像素的颜色。

本文以 hw/display/100ask_qemu_fb.c 为例进行讲解。

6.1 创建 GUI Console

创建 GUI Console 的函数有 2 个：graphic_console_init、graphic_console_init_hidden。

后者是我们添加的，它创建的 GUI Console 默认是隐藏的，要显示的话需要在“Device Manager”中点击对应的按钮。

LCD 代码如下，主要是 graphic_console_init 函数：

```
static void ask100fb_realize(DeviceState *dev, Error **errp)
{
    ASK100FbState *s = ASK100FB(dev);

    if (imx6ul_board_ui_background_prepare())
        return;

    s->invalidate = 1;
    s->fb_xres = imx6ul_board_descs[selected_board].lcd.w;
    s->fb_yres = imx6ul_board_descs[selected_board].lcd.h;
    s->board_xres = board_mem_pixels.iWidth;
    s->board_yres = board_mem_pixels.iHeight;
    dev->id = "board";
    s->con = graphic_console_init(dev, 0, &ask100fb_ops, s);
    qemu_console_resize(s->con, s->board_xres, s->board_yres);
}
```

1. 设置ID, 这会在device_manager中显示出来

2. 初始化GUI Console

3. 调整大小

调用 graphic_console_init 时，要传入一个 GraphicHwOps 结构体，如下：

```
static const GraphicHwOps ask100fb_ops = {
    .invalidate = ask100fb_invalidate,
    .gfx_update = ask100fb_update,
};
```

被周期性的调用，用来更新界面

6.2 GUI 更新函数

ask100fb_update 函数被 GUI 系统周期性地调用：

```
static void ask100fb_update(void *opaque)
{
    .....
    if (inited)
    {
        if (!s->fb_base_phys)
            return;

        //if (s->invalidate) {
            framebuffer_update_memory_section(&s->fbsection, sysbus_address_space(sbd), s->fb_base_phys,
                                                s->fb_yres, src_width);
        //}

        framebuffer_update_display(surface, &s->fbsection, s->fb_xres, s->fb_yres,
                                    src_width, dest_width, 0, 1, ask100fb_draw_line_src16,
                                    s, &first, &last);
        fb_x = imx6ul_board_descs[selected_board].lcd.x;
        fb_y = imx6ul_board_descs[selected_board].lcd.y;
        dpy_gfx_update(s->con, fb_x, fb_y, s->fb_xres, s->fb_yres);
        //}

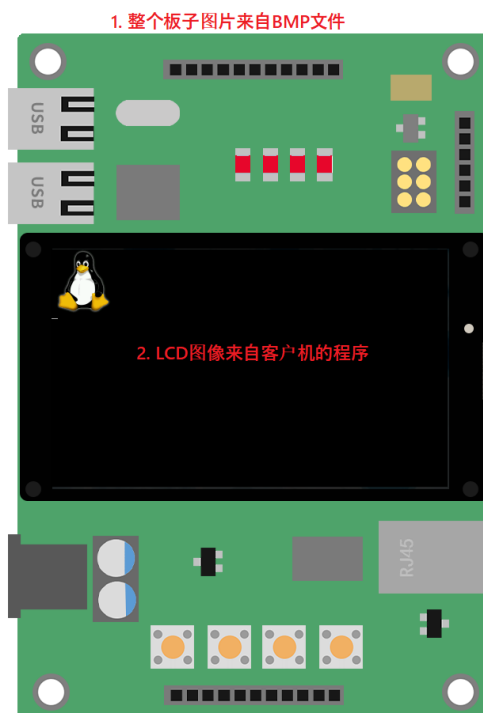
        imx_gpio_ui_update(opaque);
    } « end if inited »
    else
    {
        //dpy_gfx_update_image(s->con, "/home/book/board.bmp", 0, 0, s->board_xres, s->board_yres);
        framebuffer_update_region(surface, &board_mem_pixels, 0, 0, s->board_xres, s->board_yres);
        dpy_gfx_update(s->con, 0, 0, s->board_xres, s->board_yres);

        imx_gpio_ui_init();
    }
}
```

2. 以后：从LCD驱动程序的framebuffer中获得数据，用来更新GUI

第一次：显示一个BMP文件，这是单板的图片

效果如下：



6.3 GUI 函数详解

6.3.1 解析 BMP 文件得到 RGB 数据

```
static int imx6ul_board_ui_backgroud_prepare(void)
{
    int err;

    char *cur_app_abs_dir = get_cur_app_abs_dir(); 1. 获得APP绝对路径

    PT_PicFileParser pBMPParser = GetBMPParserInit(); 2. 获得BMP解析器
    T_FileMap tFileMap;

    /* ../../bin/./etc/xxx.bmp */ 3. 设置BMP文件名
    sprintf(tFileMap.strFileName, "%s/./etc/%s", cur_app_abs_dir, imx

    err =MapFile(&tFileMap); 4. 打开文件

    if (err)
        return -1;

    board_mem_pixels.iBpp = 32; /* PIXMAN_x8r8g8b8 */
    err = pBMPParser->GetPixelDatas(&tFileMap, &board_mem_pixels); 5. BMP转为32BPP的RGB数据

    UnMapFile(&tFileMap); 6. 解析BMP文件, 获得RGB数据

    return err; 7. 关闭文件
} « end imx6ul_board_ui_backgroud_prepare »
```

6.3.2 显示 RGB 数据

```
void framebuffer_update_region(
    DisplaySurface *ds, 1. 里面含有RGB数据
    PT_PixelDatas rgb_pixels,
    int x, /* left upper point: (x, y). */
    int y,
    int width,
    int height)
{
    uint8_t *dest;
    uint8_t *src;
    int i;
    int dest_row_pitch;
    int width_bytes;

    src = rgb_pixels->aucPixelDatas;

    dest_row_pitch = surface_stride(ds);
    dest = surface_data(ds); 2. 得到GUI的显存
    dest += y * dest_row_pitch + x * (surface_bits_per_pixel(ds) >> 3);
    width_bytes = width * (surface_bits_per_pixel(ds) >> 3);

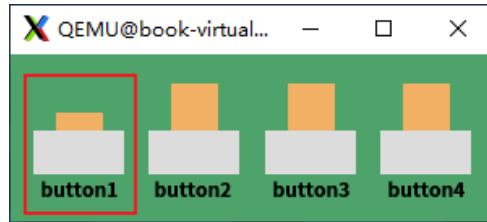
    for (i = 0; i < height; i++) {
        memcpy(dest, src, width_bytes); /* 简化版本, 因为src BPP == dest BPP */
        src += rgb_pixels->iLineBytes; 3. 使用RGB数据修改GUI显存
        dest += dest_row_pitch;
    }
} « end framebuffer_update_region »
```

6.3.3 显示 LCD 数据

```
// }
1. 含有GUI显存地址
framebuffer_update_display(surface, &s->fbsection, s->fb_xres, s->fb_yres,
    src_width, dest_width, 0, 1, ask100fb_draw_line_src16,
    s, &first, &last); 2. 含有LCD FrameBuffer地址
3. 绘制一行的函数
```

7. QEMU 的输入：鼠标事件

点击 buttons 界面时，对应的按钮会被按下或松开：



这是怎么实现的？

我们需要给这个 GUI 界面添加鼠标处理函数，代码在 ui/button_ui.c 中。这是新加的文件，要把它编进 QEMU 中需要修改同目录下的 Makefile.objs，比如修改 ui/Makefile.objs，添加一行：

```
common-obj-$(CONFIG_FSL_IMX6UL) += button_ui.o
```

7.1 分配、设置、注册 QemuInputHandler

```
static QemuInputHandler button_mouse_handler = { 1. 设置一个QemuInputHandler
    .name = "buttons",
    .mask = INPUT_EVENT_MASK_BTN | INPUT_EVENT_MASK_ABS, 能处理哪些事件？按键、绝对位移事件
    .event = button_mouse_input_event, ← 处理函数
    //.sync = hid_pointer_sync;
};

void button_ui_create(DeviceState *dev)
{
    ButtonState *bs = BUTTON_STATE(dev);

    bs->bd = buttons_desc;

    if (!button_ui_backgroud_prepare(dev))
    {
        dev->id = "buttons";
        bs->con = graphic_console_init_hidden(dev, 0, &button_ui_ops, bs);
        qemu_console_resize(bs->con, button_mem_pixels.iwidth, button_mem_pixels.iHeight);

        button_ui_show_backgroud(bs);
    }

    bs->s = qemu_input_handler_register(dev, &button_mouse_handler); 2. 注册QemuInputHandler
    qemu_input_handler_bind(bs->s, dev->id, 0, NULL); ← 3. 把设备和QemuInputHandler绑定
    qemu_input_handler_activate(bs->s); ← 4. 激活QemuInputHandler
}
```

7.2 QemuInputHandler 处理函数分析

```
static void button_mouse_input_event(DeviceState *dev, QemuConsole *src,
                                     InputEvent *evt)
{
    ButtonState *bs = BUTTON_STATE(dev);
    InputMoveEvent *move;
    InputBtnEvent *btn;
    DisplaySurface *surface;
    int scale;

    switch (evt->type) {
    case INPUT_EVENT_KIND_ABS:
        move = evt->u.abs.data;

        if (!src) {
            return;
        }
        surface = qemu_console_surface(src);
        switch (move->axis) {
            case INPUT_AXIS_X:
                scale = surface_width(surface) - 1;
                break;
            case INPUT_AXIS_Y:
                scale = surface_height(surface) - 1;
                break;
            default:
                scale = 0x8000; // 记录鼠标位置
                break;
        }
        bs->axis[move->axis] = move->value * scale / 0x7fff;
        // 转换为窗口坐标
        break;

    case INPUT_EVENT_KIND_BTN:
        btn = evt->u.btn.data;
        bs->btns[btn->button] = btn->down;
        bs->btnc[btn->button] = true;

        switch (btn->button) {
            case INPUT_BUTTON_LEFT:
                if (!btn->down) {
                    // 当鼠标左键松开时,调用
                    button_handle_button_event(dev, bs->axis[INPUT_AXIS_X], bs->axis[INPUT_AXIS_Y]);
                }
                break;
            default:
                break;
        }

        break;

    default:
        /* keep gcc happy */
        break;
    }
} // end button_mouse_input_event
```

7.3 输入事件处理流程

输入事件的处理源头是 sdl2_2d_refresh 函数:

```
static const DisplayChangeListenerOps dcl_2d_ops = {
    .dpy_name = "sdl2-2d",
    .dpy_gfx_update = sdl2_2d_update,
    .dpy_gfx_switch = sdl2_2d_switch,
    .dpy_gfx_check_format = sdl2_2d_check_format,
    .dpy_refresh = sdl2_2d_refresh, // 这里
    .dpy_mouse_set = sdl_mouse_warp,
    .dpy_cursor_define = sdl_mouse_define,
    .dpy_gfx_update_image = sdl2_2d_update_image, /* 100ask */
    .dpy_gfx_hide_or_show = sdl2_2d_dpy_gfx_hide_or_show, /* 100ask */
};
```

sdl2_2d_refresh

```
sdl2_poll_events(scon);
SDL_PollEvent(ev)
    handle_keydown(ev);
    handle_keyup(ev);
    handle_textinput(ev);
    handle_mousemotion(ev);
    handle_mousebutton(ev); // SDL_MOUSEBUTTONDOWN,UP
    sdl_send_mouse_event(scon, 0, 0, bev->x, bev->y, buttonstate);
```

7.4 QEMU 捕获的输入事件，可以作为 ABS 也可以作为 REL 事件上传

作为哪类事件上传，取决于 qemu_input_is_absolute() 的返回值：

```
-static void sdl_send_mouse_event(struct sdl2_console *scon, int dx, int dy,
                                int x, int y, int state)
{
    static uint32_t bmap[INPUT_BUTTON_MAX] = {
        [INPUT_BUTTON_LEFT] = SDL_BUTTON(SDL_BUTTON_LEFT),
        [INPUT_BUTTON_MIDDLE] = SDL_BUTTON(SDL_BUTTON_MIDDLE),
        [INPUT_BUTTON_RIGHT] = SDL_BUTTON(SDL_BUTTON_RIGHT),
    };
    static uint32_t prev_state;

    if (prev_state != state) {
        qemu_input_update_buttons(scon->dcl.con, bmap, prev_state, state);
        prev_state = state;
    }

    if (qemu_input_is_absolute()) {
        qemu_input_queue_abs(scon->dcl.con, INPUT_AXIS_X,
                            x, 0, surface_width(scon->surface));
        qemu_input_queue_abs(scon->dcl.con, INPUT_AXIS_Y,
                            y, 0, surface_height(scon->surface));
    } else {
        if (guest_cursor) {
            x -= guest_x;
            y -= guest_y;
            guest_x += x;
            guest_y += y;
            dx = x;
            dy = y;
        }
        qemu_input_queue_rel(scon->dcl.con, INPUT_AXIS_X, dx);
        qemu_input_queue_rel(scon->dcl.con, INPUT_AXIS_Y, dy);
    }
    qemu_input_event_sync();
}
```

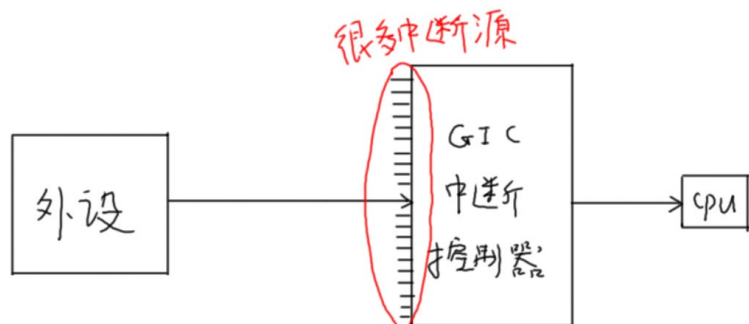
qemu_input_is_absolute 函数只会判断第 1 个 GUI Console 中的 QemuInputHandler。我们的第 1 个 GUI Console 是“Device Manager”，它的代码是 hw/display/device_manager.c。对应的 QemuInputHandler 的 mask 必须设置为 INPUT_EVENT_MASK_ABS：

```
static QemuInputHandler devmgr_mouse_handler = {
    .name = "device manager",
    .mask = INPUT_EVENT_MASK_BTN | INPUT_EVENT_MASK_ABS,
    .event = devmgr_mouse_input_event,
    //.sync = hid_pointer_sync,
};
```

↑
必须是ABS

8. 使用中断

在 QEMU 中模拟一个外设时，这个外设要使用中断的话，并不复杂。
先看看硬件框架：



8.1 外设发出哪一个中断？

GIC 从多个中断源获得中断信号，它会发信号给 CPU，这样 CPU 才会处理中断。
作为外设备，它要发出哪一个中断呢？

在 include/hw/arm/fsl-imx6ul.h 中列出了各个中断源，如下：

```
enum FslIMX6ULIRQs {
    FSL_IMX6UL_IOMUXC_IRQ    = 0,
    FSL_IMX6UL_DAP_IRQ       = 1,
    FSL_IMX6UL_SDMA_IRQ      = 2,
    FSL_IMX6UL_TSC_IRQ       = 3,
    FSL_IMX6UL_SNVS_IRQ      = 4,
    FSL_IMX6UL_LCDIF_IRQ     = 5,
    FSL_IMX6UL_BEE_IRQ       = 6,
    FSL_IMX6UL_CSI_IRQ       = 7,
    FSL_IMX6UL_PXP_IRQ       = 8,
    FSL_IMX6UL_SCTR1_IRQ     = 9,
    FSL_IMX6UL_SCTR2_IRQ     = 10,
    FSL_IMX6UL_WDOG3_IRQ     = 11,
    FSL_IMX6UL_APBH_DMA_IRQ  = 13,
    FSL_IMX6UL_WEIM_IRQ      = 14,
    FSL_IMX6UL_RAWNAND1_IRQ  = 15,
    FSL_IMX6UL_RAWNAND2_IRQ  = 16,
    FSL_IMX6UL_UART6_IRQ     = 17,
    FSL_IMX6UL_SRTC_IRQ      = 19,
    FSL_IMX6UL_SRTC_SEC_IRQ  = 20,
    FSL_IMX6UL_CSU_IRQ       = 21,
    FSL_IMX6UL_USDHC1_IRQ    = 22,
    FSL_IMX6UL_USDHC2_IRQ    = 23,
    FSL_IMX6UL_SAI3_IRQ      = 24,
    FSL_IMX6UL_SAI32_IRQ     = 25,
```

你需要查看这些中断源，确定你的外设要发出哪一个中断。
你的外设，要跟中断控制器建立联系，即确定使用哪一个中断。

8.8.1 调用 sysbus_init_irq 初始化 qemu_irq

在 QEMU 中，用 qemu_irq 结构体来描述中断，需要初始化 qemu_irq 结构体。
以 GPIO 为例，代码在 hw/gpio/imx_gpio.c 中：

```
static void imx_gpio_realize(DeviceState *dev, Error **errp)
{
    IMXGPIOState *s = IMX_GPIO(dev);

    memory_region_init_io(&s->iomem, OBJECT(s), &imx_gpio_ops, s,
                          TYPE_IMX_GPIO, IMX_GPIO_MEM_SIZE);

    qdev_init_gpio_in(DEVICE(s), imx_gpio_set, IMX_GPIO_PIN_COUNT);
    qdev_init_gpio_out(DEVICE(s), s->output, IMX_GPIO_PIN_COUNT);

    sysbus_init_irq(SYS_BUS_DEVICE(dev), &s->irq[0]); 只是初始化而已
    sysbus_init_irq(SYS_BUS_DEVICE(dev), &s->irq[1]); 还不能使用
    sysbus_init_mmio(SYS_BUS_DEVICE(dev), &s->iomem);
}
```

sysbus_init_irq 函数很有意思：

- ① 它会给设备 dev 添加一个属性 prop，prop 的名字是“sysbus-irq[0]”或“sysbus-irq[1]”
- ② 会给这个 prop 添加一个 link，link 到 &s->irq[0] 或 &s->irq[1]

8.8.2 调用 sysbus_connect_irq 连接中断

在 hw/arm/fsl-imx6ul.c 中，给 GPIO 模块和 GIC 建立联系：

```
static const int FSL_IMX6UL_GPIO_LOW_IRQ[FSL_IMX6UL_NUM_GPIO] = {
    FSL_IMX6UL_GPIO1_LOW_IRQ,
    FSL_IMX6UL_GPIO2_LOW_IRQ,
    FSL_IMX6UL_GPIO3_LOW_IRQ,
    FSL_IMX6UL_GPIO4_LOW_IRQ,
    FSL_IMX6UL_GPIO5_LOW_IRQ,
};

static const int FSL_IMX6UL_GPIO_HIGH_IRQ[FSL_IMX6UL_NUM_GPIO] = {
    FSL_IMX6UL_GPIO1_HIGH_IRQ,
    FSL_IMX6UL_GPIO2_HIGH_IRQ,
    FSL_IMX6UL_GPIO3_HIGH_IRQ,
    FSL_IMX6UL_GPIO4_HIGH_IRQ,
    FSL_IMX6UL_GPIO5_HIGH_IRQ,
};

object_property_set_bool(OBJECT(&s->gpio[i]), true, "realized",
                          &error_abort);

sysbus_mmio_map(SYS_BUS_DEVICE(&s->gpio[i]), 0,
                FSL_IMX6UL_GPIO_ADDR[i]); 把这个dev的irq[0]连接到这个中断

sysbus_connect_irq(SYS_BUS_DEVICE(&s->gpio[i]), 0,
                  qdev_get_gpio_in(DEVICE(&s->a7mpcore),
                                   FSL_IMX6UL_GPIO_LOW_IRQ[i]));
sysbus_connect_irq(SYS_BUS_DEVICE(&s->gpio[i]), 1,
                  qdev_get_gpio_in(DEVICE(&s->a7mpcore),
                                   FSL_IMX6UL_GPIO_HIGH_IRQ[i]));
```

sysbus_connect_irq 也很神奇，它是通过名字“sysbus-irq[0]”或“sysbus-irq[1]”找到设备的属性。

然后根据属性的 link 找到之前传入的 &s->irq[0] 或 &s->irq[1]，最后设置它。

设置为什么呢？下面的函数会返回一个 qemu_irq，第 2 个参数用来指定是哪一个中断：

```
qdev_get_gpio_in(DEVICE(&s->a7mpcore), FSL_IMX6UL_I2Cn_IRQ[i])
```

8.2 中断触发

发出中断：

```
qemu_set_irq(irq, 1);
```

清除中断：

```
qemu_set_irq(irq, 0);
```

上面的 2 个调用必须成双出现，如果多次发出中断而没有清除中断，那后面发出的中断是无效的。

这里说的成双出现，并不是说它们必须前后出现。流程是这样的：

- ① 用户按下按键后，使用“qemu_set_irq(irq, 1)”发出中断
- ② 客户机的程序处理完中断后，它会写 ISR 寄存器来清除中断：

对 ISR 寄存器的写操作会导致 QEMU 中对应的 write 函数被调用，在里面执行“qemu_set_irq(irq, 0)”。

对于 hw/gpio/imx_gpio.c，发出中断的函数流程为：

```
notify_imx_gpio_change
imx_gpio_set(s, pin, level);
    imx_gpio_update_int(s);
        qemu_set_irq(s->irq[0], 1);
```

清除中断的流程为：

```
imx_gpio_write
case ISR_ADDR:
    s->isr &= ~value;
    imx_gpio_set_all_int_lines(s);
        qemu_set_irq(s->irq[0], 0);
```

9. 模拟按键

我们模拟的按键涉及 GUI、鼠标输入、中断，值得仔细研究。

GUI 和鼠标的处理，都在这个文件里：ui/button_ui.c

当用户在 GUI 中点击某个按键时，它对应哪个引脚？

这由这个文件决定：hw/gpio/100ask_imx6ull_buttons.c

上述文件确定是哪一个引脚之后，要通知 hw/gpio/imx_gpio.c 来处理。

9.1 按键 GUI

```
static QemuInputHandler button_mouse_handler = {
    .name = "buttons",
    .mask = INPUT_EVENT_MASK_BTN | INPUT_EVENT_MASK_ABS,
    .event = button_mouse_input_event,
    //.sync = hid_pointer_sync,
};

void button_ui_create(DeviceState *dev)
{
    ButtonState *bs = BUTTON_STATE(dev);

    bs->bd = buttons_desc;

    if (!button_ui_backgroud_prepare(dev))
    {
        dev->id = "buttons";
        bs->con = graphic_console_init_hidden(dev, 0, &button_ui_ops, bs);
        qemu_console_resize(bs->con, button_mem_pixels.iWidth, button_mem_pixels.iHeight);
        button_ui_show_backgroud(bs);
    }

    bs->s = qemu_input_handler_register(dev, &button_mouse_handler);
    qemu_input_handler_bind(bs->s, dev->id, 0, NULL);
    qemu_input_handler_activate(bs->s);
}
```

1. 创建GUI Console

2. 注册输入处理函数

9.2 点击按键

```
static void button_mouse_input_event(DeviceState *dev, QemuConsole *src,
                                     InputEvent *evt)
{
    ButtonState *bs = BUTTON_STATE(dev);
    InputMoveEvent *move;
    InputBtnEvent *btn;
    DisplaySurface *surface;
    int scale;

    switch (evt->type) {
    case INPUT_EVENT_KIND_ABS:
        move = evt->u.abs.data;

        if (!src) {
            return;
        }
        surface = qemu_console_surface(src);
        switch (move->axis) {
            case INPUT_AXIS_X:
                scale = surface_width(surface) - 1;
                break;
            case INPUT_AXIS_Y:
                scale = surface_height(surface) - 1;
                break;
            default:
                scale = 0x8000; 记录鼠标位置
                break;
        }
        bs->axis[move->axis] = move->value * scale / 0x7fff;
        break;

    case INPUT_EVENT_KIND_BTN:
        btn = evt->u.btn.data;
        bs->btns[btn->button] = btn->down;
        bs->btnc[btn->button] = true;

        switch (btn->button) {
            case INPUT_BUTTON_LEFT:
                if (!btn->down) 当鼠标左键松开时,调用
                {
                    button_handle_button_event(dev, bs->axis[INPUT_AXIS_X], bs->axis[INPUT_AXIS_Y]);
                }
                break;
            default:
                break;
        }

        break;

    default:
        /* keep gcc happy */
        break;
    }
}
/* end button_mouse_input_event */
```

转换为窗口坐标

```
static void button_handle_button_event(DeviceState *dev, int x, int y)
{
    int index = get_button_index_for_xy(dev, x, y); 1. 根据鼠标位置确定按钮
    if (index == -1)
        return;
    button_update_item(dev, index); 2. 更新按钮图片
}
```

9.3 更新按键图片的同时

```
static void button_update_item(void *opaque, int index)
{
    ButtonState *bs = BUTTON_STATE(opaque);
    button_desc *bd = bs->bd;
    DisplaySurface *surface = qemu_console_surface(bs->con);
    int x, y;

    x = bd[index].x;
    y = bd[index].y;

    bd[index].pressed = !bd[index].pressed;

    notify_board_button_change(index); 1. 通知单板代码处理按键

    if (bd[index].pressed) 2. 更新按钮图片
    {
        framebuffer_update_region(surface, &button_pressed_pixels, x, y, button_pressed_pixels.iWidth,
    }
    else
    {
        framebuffer_update_region(surface, &button_released_pixels, x, y, button_released_pixels.iWidth,
    }

    dpy_gfx_update(bs->con, 0, 0, button_mem_pixels.iWidth, button_mem_pixels.iHeight);
}
/* end button_update_item */
```

9.4 单板的代码怎么处理按键

对应的代码在 hw/gpio/100ask_imx6ull_buttons.c 中:

```
void notify_board_button_change(int index) ← 哪个按键?
{
    int group = pin_to_button_map[index][0]; 确定是哪个引脚
    int pin = pin_to_button_map[index][1]; 确定是否按下
    int level = is_button_pressed(index);

    notify_imx_gpio_change(group, pin, level); 通知imx6ull的代码处理
}
```

9.5 IMX6ULL 的代码怎么处理输入引脚

对应的代码在 hw/gpio/imx_gpio.c 中：

```
void notify_imx_gpio_change(int group, int pin, int level)
{
    char path[100];
    sprintf(path, "/machine/soc/gpio%d", group-1);
    IMXGPIOState *s = IMX_GPIO(object_resolve_path(path, NULL));

    if (s)
    {
        imx_gpio_set(s, pin, level);
    }
}

static void imx_gpio_set(void *opaque, int line, int level)
{
    IMXGPIOState *s = IMX_GPIO(opaque);
    IMXGPIOLevel imx_level = level ? IMX_GPIO_LEVEL_HIGH : IMX_GPIO_LEVEL_LOW;

    imx_gpio_set_int_line(s, line, imx_level);

    /* this is an input signal, so set PSR */
    s->psr = deposit32(s->psr, line, 1, imx_level);
    imx_gpio_update_int(s);
}
```

1. 更新psr寄存器
客户机程序可以读到引脚值

2. 发出中断

10. QEMU 启动过程

10.1 各个模块的注册

各个模块都定义了一个 TypeInfo 结构体，比如 LCD：

```
static const TypeInfo ask100fb_info = {
    .name       = TYPE_100ASKFB,
    .parent     = TYPE_SYS_BUS_DEVICE,
    .instance_size = sizeof(ASK100FbState), /* object_new_with_type 创建设备时会malloc该结构体，并传给instance_init */
    .class_init  = ask100fb_class_init, /* object_init_with_type */
    .instance_init = ask100fb_init,
};

void create_100ask_qemu_fb(void)
{
    DeviceState *dev;

    dev = qdev_create(NULL, TYPE_100ASKFB);
    qdev_init_nofail(dev);

    static void ask100fb_register_types(void)
    {
        type_register_static(&ask100fb_info);
    }
    type_init(ask100fb_register_types)
}
```

4. 创建设备时，会分配ASK100FbState结构体
2. 调用此函数
3. 创建qdev，传入名字
5. 初始化实例
6. 初始化类别

并且都使用 type_init 定义了一个属性为 **constructor** 的函数(include/qemu/module.h)：

```
#define type_init(function) module_init(function, MODULE_INIT_QOM)
#define module_init(function, type)
static void __attribute__((constructor)) do_qemu_init_## function(void)
{
    register_dso_module_init(function, type);
}

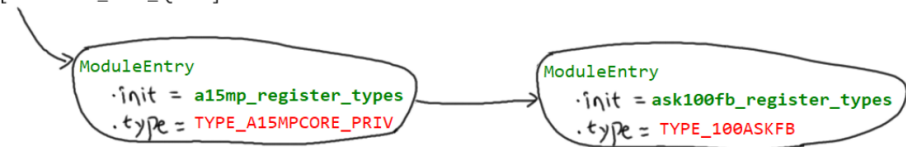
void register_dso_module_init(void (*fn)(void), module_init_type type)
{
    ModuleEntry *e;

    init_lists();

    e = g_malloc0(sizeof(*e));
    e->init = fn;
    e->type = type;
    QTAILQ_INSERT_TAIL(&dso_init_list, e, node);
}
```

这些属性为 **constructor** 的函数会在 main 函数之前被调用，这样就得到了一个链表：

init_type_list[MODULE_INIT_QOM]



10.2 各个模块的初始化

main 函数位于 vl.c 中，它有如下调用：

```
main
module_call_init(MODULE_INIT_QOM);
QTAILQ_FOREACH(e, l, node) {
    e->init();
}
```

这些 xxx_register_types 执行后，又得到了什么？

- ① 分配一个 TypeImpl 结构体，使用 TypeInfo 来设置它：

```
static TypeImpl *type_new(const TypeInfo *info)
{
    TypeImpl *ti = g_malloc0(sizeof(*ti));
    int i;

    g_assert(info->name != NULL);

    if (type_table_lookup(info->name) != NULL) {
        fprintf(stderr, "Registering '%s' which already\n", info->name);
        abort();
    }

    ti->name = g_strdup(info->name);
    ti->parent = g_strdup(info->parent);

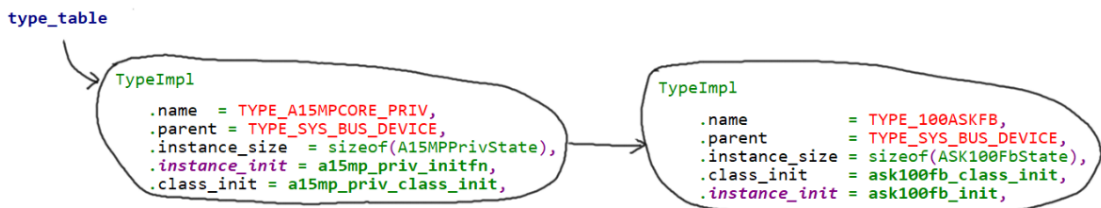
    ti->class_size = info->class_size;
    ti->instance_size = info->instance_size;

    ti->class_init = info->class_init;
    ti->class_base_init = info->class_base_init;
    ti->class_data = info->class_data;

    ti->instance_init = info->instance_init;
```

- ② 把 TypeImpl 放入链表：type_table

于是，在程序中就有了这样的链表：



程序中有一系列的 TypeImpl，但是并不表示会用到它们。要用某个 TypeImpl，需要创建对应的设备。

10.3 选择 machine

启动 qemu 时, 会传入参数“-M mcimx6ul-evk”, 对应的文件为: hw/arm/mcimx6ul-evk.c

```
static void mcimx6ul_evk_machine_init(MachineClass *mc)
{
    mc->desc = "Freescale i.MX6UL Evaluation Kit (Cortex A7)";
    mc->init = mcimx6ul_evk_init;
    mc->max_cpus = FSL_IMX6UL_NUM_CPUS;
}
DEFINE_MACHINE("mcimx6ul-evk", mcimx6ul_evk_machine_init)
```

10.4 选择 CPU

```
static void mcimx6ul_evk_init(MachineState *machine)
{
    static struct arm_boot_info boot_info;
    MCIMX6ULEVK *s = g_new0(MCIMX6ULEVK, 1);
    int i;

    if (machine->ram_size > FSL_IMX6UL_MMDC_SIZE) {
        error_report("RAM size " RAM_ADDR_FMT " above max supported (%08x)",
                     machine->ram_size, FSL_IMX6UL_MMDC_SIZE);
        exit(1);
    }

    boot_info = (struct arm_boot_info) {
        .loader_start = FSL_IMX6UL_MMDC_ADDR,
        .board_id = -1,
        .ram_size = machine->ram_size,
        .nb_cpus = machine->smp_cpus,
    };

    object_initialize_child(OBJECT(machine), "soc", &s->soc, sizeof(s->soc),
        CPU → TYPE_FSL_IMX6UL, &error_fatal, NULL);
}
```

CPU 在中 hw/arm/fsl-imx6ul.c 定义:

```
static const TypeInfo fsl_imx6ul_type_info = {
    .name = TYPE_FSL_IMX6UL,
    .parent = TYPE_DEVICE,
    .instance_size = sizeof(FslIMX6ULState),
    .instance_init = fsl_imx6ul_init,
    .class_init = fsl_imx6ul_class_init,
};

static void fsl_imx6ul_register_types(void)
{
    type_register_static(&fsl_imx6ul_type_info);
}
type_init(fsl_imx6ul_register_types)
```

10.5 CPU 的初始化

hw/arm/mcimx6ul-evk.c mcimx6ul_evk_init:

```
object_initialize_child(OBJECT(machine), "soc", &s->soc, sizeof(s->soc),
    TYPE_FSL_IMX6UL, &error_fatal, NULL);

object_property_set_bool(OBJECT(&s->soc), true, "realized", &error_fatal);
```

这会导致 hw/arm/fsl-imx6ul.c 中 fsl_imx6ul_init 函数被调用:

```
static const TypeInfo fsl_imx6ul_type_info = {
    .name = TYPE_FSL_IMX6UL,
    .parent = TYPE_DEVICE,
    .instance_size = sizeof(FslIMX6ULState),
    .instance_init = fsl_imx6ul_init,
    .class_init = fsl_imx6ul_class_init,
};

static void fsl_imx6ul_register_types(void)
{
    type_register_static(&fsl_imx6ul_type_info);
}

type_init(fsl_imx6ul_register_types)
```

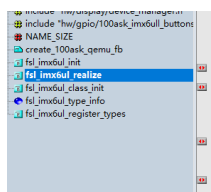
在 fsl_imx6ul_init 函数中, 会去创建名为 a7mpcore 的设备, 它的类型是 TYPE_A15MPCORE_PRIV:

```
static void fsl_imx6ul_init(Object *obj)
{
    FslIMX6ULState *s = FSL_IMX6UL(obj);
    char name[NAME_SIZE];
    int i;

    object_initialize_child(obj, "cpu0", &s->cpu, sizeof(s->cpu),
        ARM_CPU_TYPE_NAME("cortex-a7"), &error_abort, NULL);

    /*
     * A7MPCORE
     */
    sysbus_init_child_obj(obj, "a7mpcore", &s->a7mpcore, sizeof(s->a7mpcore),
        TYPE_A15MPCORE_PRIV);

    /*
     * A7MPCORE
     */
    object_property_set_int(OBJECT(&s->a7mpcore), 1, "num-cpu", &error_abort);
    object_property_set_int(OBJECT(&s->a7mpcore),
        FSL_IMX6UL_MAX_IRQ + GIC_INTERNAL,
        "num-irq", &error_abort);
    object_property_set_bool(OBJECT(&s->a7mpcore), true, "realized",
        &error_abort);
    sysbus_mmio_map(SYS_BUS_DEVICE(&s->a7mpcore), 0, FSL_IMX6UL_A7MPCORE_ADDR);
}
```



[a15mpcore.c (qemu\hw\cpu)]
is Tools View Window Help

ore.h (qemu\include\hw\cpu) boot.c (qemu\hw\arm) fsl-imx6ul.c (qemu\hw\arm) mcimx6ul-evk.c (qemu

```
170:
171: static const TypeInfo a15mp_priv_info = {
172:     .name = TYPE_A15MPCORE_PRIV,
173:     .parent = TYPE_SYS_BUS_DEVICE,
174:     .instance_size = sizeof(A15MPPrivState),
175:     .instance_init = a15mp_priv_initfn,
176:     .class_init = a15mp_priv_class_init,
177: };
178:
```

a7mpcore 设备对应的 realize 函数被调用时, 里面会实例化 GIC 等。

10.6 在 system memory 中添加内存

分配 system memory，代码为 hw/arm/mcimx6ul-evk.c 中的 mcimx6ul_evk_init 函数：

```
memory_region_allocate_system_memory(&s->ram, NULL, "mcimx6ul-evk.ram",  
                                     machine->ram_size);  
memory_region_add_subregion(get_system_memory(),  
                             内存基地址 → FSL_IMX6UL_MMDC_ADDR, &s->ram);
```

参数指定

其中的 ram_size 来自 QEMU 运行时的参数“-m 512M”；内存基地址为 0x80000000。

从这里可以知道，system memory 中既含有可读可写的内存，也含有各种模块的寄存器。

11. 调试

11.1 要调试，运行 gdb 程序之前，必须先进入源码目录

无论是使用 gdb 调试 PC 程序，还是使用 arm-xxx-gdb 调试 ARM 程序，运行这些 gdb 程序前，必须进入要调试的源码的目录。

然后再启动 gdb 或 arm-xxx-gdb。

11.2 调试 QEMU 本身

先进入 QEMU 源码目录，执行如下命令。

这个命令中涉及 qemu-system-arm、zImage、设备树、文件系统这 4 个文件的路径，你要改成你的路径。

```
gdb --args /home/book/mywork/qemu/qemu_git/qemu/bin/qemu-system-arm -M \
mcimx6ul-evk -m 512M -kernel /home/book/nfs_rootfs/zImage \
-dtb /home/book/nfs_rootfs/mynet.dtb -serial stdio \
-drive
file=/home/book/mywork/imx6ull_test_image/buildroot_rootfs.img,format=raw,id=mysdcard -device sd-card,drive=mysdcard \
-append "console=ttyMXC0,115200 console=tty1 rootfstype=ext4 root=/dev/mmcblk1
rw rootwait init=/sbin/init loglevel=8" \
-nic user
```

关键点在于：gdb --args，它后面就可以跟一连串的命令及参数了。

这种方法有一个缺点，就是客户机上的 Linux 系统启动后，你无法再在终端中输入 gdb 命令。可以使用下一种方法来调试。

11.3 调试之前就运行的 QEMU

先进入 QEMU 源码目录。

① 首先确定 qemu-system-arm 进程的 PID

执行如下命令：

```
ps -A | grep qemu
```

假设得到 PID 为 9527

② 在 QEMU 源码目录下执行如下命令：

```
sudo gdb -p 9527
```

注意：一定要加上 sudo

11.4 调试客户机上的 Linux 内核

11.4.1 首先运行 qemu-system-arm

执行 QEMU 的命令时，在最后加上参数“-s -S”。

-s 是“-gdb tcp::1234”的缩写，表示 QEMU 将会在本机端口 1234 上开启 gdbserver。

-S 表示不启动 CPU，当你使用 arm-xxx-gdb 连接上 QEMU 后，必须执行 c 命令才可以运行内核。

我用的命令如下所示。

这个命令中涉及 qemu-system-arm、zImage、设备树、文件系统这 4 个文件的路径，你要改成你的路径。

```
gdb --args /home/book/mywork/qemu/qemu_git/qemu/bin/qemu-system-arm -M \
mcimx6ul-evk -m 512M -kernel /home/book/nfs_rootfs/zImage \
-dtb /home/book/nfs_rootfs/mynet.dtb -serial stdio \
-drive
file=/home/book/mywork/imx6ull_test_image/buildroot_rootfs.img,format=raw,id=mysdcard \
-device sd-card,drive=mysdcard \
-append "console=ttyMXC0,115200 console=tty1 rootfstype=ext4 root=/dev/mmcblk1
rw rootwait init=/sbin/init loglevel=8" \
-nic user -s -S
```

14.4.2 进入内核源码目录，运行 arm-xxx-gdb vmlinux

```
book@book-virtual-machine:~/100ask_imx6ull-gemu/linux-4.9.88$ arm-linux-gnueabi-gdb vmlinux
GNU gdb (Linaro_GDB-2016.11) 7.12.0.20161122-git
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "--host=x86_64-unknown-linux-gnu --target=arm-linux-gnueabi".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from vmlinux...done.
(gdb) target remote 127.0.0.1:1234
Remote debugging using 127.0.0.1:1234
0x80000000 in ?? ()
(gdb) b start_kernel
Breakpoint 1 at 0x80e008e0: file init/main.c, line 486.
(gdb) c
Continuing.

Breakpoint 1, start_kernel () at init/main.c:486
486      set_task_stack_end_magic(&init_task);
(gdb)
```

调试内核vmlinux

连接QEMU的gdbserver

设置断点

运行内核